



清华大学

综合论文训练

基于多模态大语言模型的数字电路时序图解析

系 别： 计算机科学与技术系

专 业： 计算机科学与技术

姓 名： 汤 秉 达

指导教师： 喻 文 健 教 授

二〇二六年六月

关于论文使用授权的说明

本人完全了解清华大学有关保留、使用综合论文训练论文的规定，即：学校有权保留论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

作者签名：

导师签名：

日 期：

日 期：

摘 要

数字电路时序图是芯片设计与验证中编码模块行为规约的核心视觉载体，蕴含了实现对应硬件描述语言代码所需的关键逻辑信息。然而，从时序图到硬件描述语言代码的转化至今仍高度依赖工程师的手工操作，效率低下且极易引入人为错误。近年来，多模态大语言模型在跨模态理解任务中展现出卓越性能，但将其应用于时序图等高度专业化的技术图表仍面临训练数据匮乏、缺乏专用评价方法、领域视觉特征与通用图像差异显著以及生成代码的功能正确性难以保证等挑战。

本文提出了一种基于多模态大语言模型的数字电路时序图自动解析框架，将该任务建模为两阶段流水线：首先由多模态大语言模型将时序图图像转换为结构化表示，再由文本大语言模型基于结构化表示推理生成硬件描述语言代码。在数据层面，本文优化了数据合成链路，利用大语言模型替代随机激励生成高质量测试平台，并构建多层级预选机制排除不合格的候选模块，显著提升了训练数据的合理性与实用性。在评价层面，本文提出了面向时序图结构化表示的专用评价指标，解决了传统文本相似度指标无法衡量结构语义正确性的问题。在训练层面，本文首先通过有监督微调分别训练两阶段模型，建立从“图像到结构化表示”及“结构化表示到硬件描述语言代码”的基线能力；随后以上述评价指标为奖励函数，采用群体相对策略优化算法进行强化学习，进一步提升生成代码的功能正确性。实验结果表明，本文方法在时序图解析任务上显著超越开源基线模型并逼近前沿闭源模型的性能水平，为智能电子设计自动化工具的发展开辟了新的路径。

关键词：多模态大语言模型；数字电路时序图；电子设计自动化；有监督微调；强化学习

Abstract

Digital timing diagrams encode the behavioral specifications of hardware modules and are central to chip design and verification. Converting them into Hardware Description Language (HDL) code, however, still relies heavily on manual effort—a process that is both time-consuming and error-prone. While Multimodal Large Language Models (MLLMs) have shown remarkable cross-modal understanding capabilities, applying them to highly specialized technical diagrams such as timing diagrams remains challenging due to the scarcity of training data, the lack of domain-specific evaluation methods, the significant gap between domain-specific visual features and natural images, and the difficulty of ensuring functional correctness of generated code.

This paper presents an MLLM-based automated parsing framework that formulates timing diagram analysis as a two-stage pipeline: an MLLM first converts the timing diagram image into a structured intermediate representation, and a text-based LLM then reasons over this representation to generate HDL code. To address data scarcity, we optimize the data synthesis pipeline by employing LLMs to generate semantically meaningful testbenches in place of random stimuli and by introducing a multi-level pre-selection mechanism to exclude unqualified candidate modules. For evaluation, we propose a domain-specific metric tailored for structured timing representations, addressing the inability of conventional text-similarity metrics to capture structural semantic correctness. For training, we first apply Supervised Fine-Tuning (SFT) to the two-stage models separately, establishing baseline capabilities for cross-modal conversion from images to structured representations and from structured representations to HDL code; we then employ the proposed metric as a reward function in a Reinforcement Learning stage with the Group Relative Policy Optimization (GRPO) algorithm, further improving the functional correctness of the generated code. Experimental results demonstrate that the proposed method significantly outperforms open-source baseline models and approaches the performance level of state-of-the-art proprietary models on the timing diagram parsing task, opening a new pathway for the development of intelligent Electronic Design Automation tools.

Keywords: Multimodal Large Language Models; Digital Timing Diagrams; Electronic Design Automation; Supervised Fine-Tuning; Reinforcement Learning

目 录

第 1 章 引 言.....	1
1.1 研究背景	1
1.2 本文贡献	3
1.3 论文组织结构	4
第 2 章 问题背景.....	5
2.1 关键技术基础	5
2.1.1 大语言模型	5
2.1.2 多模态大语言模型	6
2.1.3 WaveJSON 表示格式	7
2.1.4 评价指标	9
2.1.5 有监督微调	9
2.1.6 强化学习	10
2.2 相关工作	10
2.2.1 大语言模型在硬件设计中的应用	10
2.2.2 时序图自动化分析	11
2.3 问题定义	12
第 3 章 方 法.....	13
3.1 框架概览	13
3.2 测试平台生成策略	14
3.2.1 基于随机激励的基线方案	14
3.2.2 基于大语言模型的优化方案	15
3.3 数据质量预选	16
3.3.1 Verilog 源码预筛	16
3.3.2 基于随机管线的仿真产出质量预选	17
3.4 防泄漏数据划分	17
3.5 评价指标	18
3.5.1 信号匹配	18
3.5.2 波形评分	18
3.5.3 综合评分	19
3.5.4 基于仿真对拍的代码评估	19

3.6 有监督微调	20
3.6.1 第一阶段：时序图图像到 WaveJSON	20
3.6.2 第二阶段：WaveJSON 到 Verilog 代码	20
3.7 基于强化学习的优化	21
3.7.1 奖励函数设计	21
3.7.2 GRPO 优化	21
第 4 章 实验结果	23
4.1 数据集统计分析	23
4.1.1 数据规模	23
4.1.2 信号数量与代码长度分布	23
4.1.3 Verilog 代码长度分布	24
4.1.4 时序图图像尺寸分布	24
4.1.5 数据划分	25
4.1.6 模块功能多样性	25
4.1.7 时序图样本示例与激励方案对比	27
4.2 评价指标有效性分析	28
4.3 第一阶段有监督微调实验结果	31
4.3.1 总体结果	31
4.3.2 解析成功样本分析	32
4.3.3 错误模式分析	33
4.4 第二阶段有监督微调实验结果	34
4.4.1 总体结果	35
4.4.2 仿真成功样本分析	36
4.4.3 仿真失败分析	37
4.4.4 逻辑实现错误分析	37
4.5 第二阶段强化学习实验结果	38
4.5.1 总体结果	38
4.5.2 结果分析	39
4.5.3 典型改善案例	40
4.6 本章小结	41
第 5 章 总结与展望	42
5.1 工作总结	42
5.2 局限性与未来展望	42

参考文献.....	44
附录 A 外文资料的书面翻译	47
致 谢.....	70
声 明.....	72

插图清单

图 2.1 WaveJSON 示例渲染效果.....	9
图 3.1 时序图自动化解析框架概览.....	13
图 4.1 样本信号数量分布.....	24
图 4.2 Verilog 代码长度分布	24
图 4.3 时序图图像的宽高比与像素总数分布.....	25
图 4.4 同一状态机模块在两种激励方案下的时序图对比.....	27
图 4.5 同一 FIFO 模块在两种激励方案下的时序图对比	28
图 4.6 数据集中不同功能类别的时序图样本.....	29
图 4.7 评价指标验证场景 1：信号顺序置换	30
图 4.8 评价指标验证场景 2：保持符号等价	30
图 4.9 评价指标验证场景 3：数据标签错误	30
图 4.10 评价指标验证场景 4：信号缺失.....	31
图 4.11 第一阶段测试集 F1 分数分布	32
图 4.12 第一阶段完全正确的复杂样本	33
图 4.13 第一阶段波形长度偏差错误示例	33
图 4.14 第一阶段数据标签识别错误示例	34
图 4.15 第二阶段测试集 F1 分数分布（SFT 模型）	36
图 4.16 第二阶段完全正确的复杂样本	36
图 4.17 GRPO 训练过程中平均 F1 与仿真通过率的变化曲线.....	39

附表清单

表 2.1 WaveJSON 波形字符及其含义.....	8
表 4.1 数据集模块功能分类统计.....	26
表 4.2 传统文本相似度指标与本文评价指标的对比.....	29
表 4.3 第一阶段不同模型的评估结果对比.....	32
表 4.4 第二阶段不同模型的评估结果对比.....	35
表 4.5 GRPO 强化学习优化前后的评估结果对比.....	38

第 1 章 引 言

1.1 研究背景

随着集成电路工艺节点的持续演进与片上系统（System on Chip, SoC）规模的急剧扩大，数字芯片的设计复杂度正以指数级增长。在这一背景下，电子设计自动化（Electronic Design Automation, EDA）工具承担着将人类设计意图转化为可制造芯片的使命。从逻辑综合到时序分析，从布局布线到物理验证，EDA 工具链已在设计流程的各环节实现了高度自动化。然而，一个长期被忽视的瓶颈在于设计流程的起点——将人类可读的视觉设计规范转化为机器可执行的硬件描述语言（Hardware Description Language, HDL）代码。这一步骤至今仍以手工完成为主。

数字电路时序图（Digital Timing Diagram）是这类视觉化设计规范中最具代表性的一种。它以时钟周期为横轴，以信号电平为纵轴，清晰地编码了数字系统的行为规约：信号之间的先后次序、组合逻辑与时序逻辑的因果关系，以及不同功能状态之间的转换条件。无论是数据手册中的通信协议时序（如 SPI、I2C、UART），还是复杂总线协议（如 AMBA AXI[1]、AHB[2]、APB[3]）的传输波形，时序图都是定义模块行为的权威依据。可以说，时序图本身就是一种“图形化的硬件规格说明书”，其中蕴含了实现对应 HDL 代码所需的关键逻辑信息。

当前的工程实践中，将时序图转化为 HDL 代码的过程完全依赖工程师的个人经验与手工劳动。工程师需要逐周期地阅读波形，在头脑中重建有限状态机（Finite State Machine, FSM）与数据通路，再将这些抽象逻辑翻译为 Verilog 或 VHDL 代码。这一过程存在两个根本性的问题：其一，效率低下——包含数十个信号、跨越数百个时钟周期的复杂协议时序图，人工解读往往需要数小时甚至更长时间；其二，容易出错——信号间的细微依赖关系与边界条件极易被忽略，由此引发的设计缺陷通常要到仿真甚至流片阶段才被暴露，造成高昂的修复成本。因此，如果能够自动地从时序图图像中提取结构化的时序信息，并据此生成功能正确的 HDL 代码，将从根本上改变芯片设计的前端范式。

近年来，深度学习技术的飞速发展为此问题带来了全新的解决思路。大语言模型（Large Language Model, LLM）以 Transformer 架构 [4] 为基础，以 GPT-5.5[5] 等闭源模型与 LLaMA 4[6]、Qwen3[7]、DeepSeek-V4[8] 等开源模型为代表经历了快速迭代，在自然语言理解、代码生成等任务上已展现出接近甚至超越人类的表现。在硬件设计领域，已有多项工作利用 LLM 实现从文本需求到 Verilog 代码的自动生成 [9-15]，证明了 LLM 在理解硬件语义方面的潜力。与此同时，多模态大

语言模型（Multimodal Large Language Model, MLLM）的兴起进一步打破了模态壁垒。GPT-5[16]、Gemini 2.5[17] 等闭源模型，以及 LLaVA[18]、InternVL3.5[19]、LLaMA 4[6]、Qwen3-VL[20] 等开源模型，已在自然图像理解、文档解析、图表推理等任务中取得了卓越成效。这些进展表明，MLLM 具备从视觉输入中提取结构化语义信息的基础能力，为实现“时序图图像到 HDL 代码”的自动化提供了技术基础。然而，将通用 MLLM 直接应用于数字电路时序图的解析面临训练数据匮乏、领域视觉特征差异显著、缺乏专用评价方法以及生成代码功能正确性难以保证等挑战。围绕这些问题，已有若干相关研究进行了探索。

He 等人 [21] 提出的 TD-Magic 能够从时序图图像中提取严格偏序关系作为形式化规范，但其基于规则的多模块设计难以扩展到复杂场景，且仅适用于包含显式偏序关系的少数时序图。同一团队后续提出的 TD-Interpreter[22] 则采用了不同的技术路线，通过对开源 MLLM LLaVA 进行微调，构建了一个面向时序图的视觉问答（Visual Question Answering, VQA）系统。TD-Interpreter 开发了包含描述型与推理型两类数据的合成生成流程，使微调后的 LLaVA 在时序图理解任务上大幅超越了未经微调的 GPT-4o。该工作证明了通过领域特定数据对 MLLM 进行微调以理解时序图的可行性，但存在以下局限：其一，定位为交互式问答工具，其输入为时序图图像与自然语言问题，输出为自然语言回答，并不涉及结构化信息的提取与 HDL 代码的生成；其二，训练数据中描述型问答过于简单（如信号数量、电平值等事实性查询），而涉及因果推理的推理型问答则依赖人工设计问题模板与标注答案，难以规模化扩展；其三，其数据合成流程采用基于随机激励的测试平台（Testbench）生成方案，仿真产生的时序图缺乏有意义的功能行为模式，限制了训练数据的有效性；其四，评价体系仅采用 BLEU、ROUGE 等通用文本相似度指标，缺乏面向时序图结构语义的专用评价方法；其五，仅采用有监督微调进行训练，未探索基于强化学习的进一步优化。此外，Chang 等人 [23] 将 MLLM 引入硬件设计流程，利用视觉与文本的多模态输入生成 Verilog 代码，但该工作并未关注时序图，其视觉输入主要面向其他类型的设计图表。

与上述工作不同，本文选择将时序图解析任务建模为从图像到结构化表示、再到 HDL 代码的生成式流水线，而非视觉问答范式。这一任务设定的选择基于以下两方面的考量。其一，在数据构建的可扩展性方面，结构化表示与 HDL 代码均可从现有的 Verilog 源代码通过仿真工具链自动获取——给定一个 Verilog 模块及其测试平台，仿真即可生成时序图图像，而模块源代码与仿真结果则天然构成对应的标注数据。这意味着整个数据构建流程可以完全自动化，无需像视觉问答任务那样依赖大量人工设计的问答对，从而具备更强的规模化能力。其二，在任务本

身的深度与实用性方面，结构化表示要求模型忠实地还原时序图中每一个信号的名称、电平序列与时钟关系，其输出可被程序化地解析与比较，从而为自动化评估提供客观基础，也为从时序图自动生成断言、检测设计文档与 RTL 实现的一致性等潜在下游应用奠定技术基础。在此基础上进一步生成 HDL 代码，本质上是对时序图所描述的硬件行为进行逆向工程——模型不仅需要“看懂”每个信号的波形，还需推理出信号之间的因果逻辑并将其转化为可综合的电路描述。因此，HDL 代码的功能正确性也构成了衡量模型对时序图理解深度的严格标准。

尽管上述任务设定在数据构建的可扩展性与评价深度方面均具有优势，但要在该框架下实现端到端的自动化解析，仍需克服一系列关键技术瓶颈。具体而言，本文所面对的核心挑战包括：第一，训练数据方面，当前不存在面向时序图解析的高质量标注数据集，且基于随机激励生成的时序图往往缺少有意义的功能模式，无法为模型训练提供有效监督信号；第二，评价体系方面，传统的文本相似度指标（如 BLEU、ROUGE）无法反映结构化时序表示的语义正确性，亟需设计专门的评价方法；第三，模型能力方面，时序图的视觉特征高度领域化，需要通过针对性的微调使 MLLM 掌握从像素到结构化时序信息的映射能力，同时还需训练文本 LLM 完成从结构化表示到可综合 HDL 代码的转换；第四，代码质量方面，仅依赖有监督学习难以保证生成代码的功能正确性，有必要探索基于仿真反馈的优化机制来闭合“生成-验证”的循环。

1.2 本文贡献

针对上述挑战，本文提出了一种基于多模态大语言模型的数字电路时序图自动化解析框架。该框架将时序图解析建模为两阶段任务：第一阶段由 MLLM 将时序图图像转换为结构化的中间表示，第二阶段由文本 LLM 将结构化表示翻译为对应的 HDL 代码。本文的主要贡献如下：

1. **优化数据合成链路，构建高质量训练数据集。**针对该领域标注数据匮乏的问题，本文对数据合成链路进行了系统优化。在数据生成端，利用大语言模型替代随机激励来生成测试平台，使仿真产生的时序图呈现出更具工程意义的信号行为模式。在数据筛选端，设计了多层级的质量预选机制，排除无法独立仿真或产出质量不合格的候选模块。在数据划分端，采用基于并查集的防泄漏划分策略，通过多维度相似性检测将高度相似的样本归入同一等价类并整组分配，避免训练集与测试集之间的信息泄漏，确保评估结果的可靠性。
2. **提出面向时序图结构化表示的专用评价指标。**鉴于 BLEU、ROUGE 等通用文本相似度指标对信号排列顺序敏感且无法区分语义关键错误的严重程度，本

文设计了基于信号级精确率-召回率的专用评价指标。该指标首先通过精确匹配与模糊匹配建立预测信号与标准答案信号之间的对应关系，然后在展开保持符号后逐周期比较波形序列的一致性，最终汇总为 F1 分数。该指标同时作为强化学习阶段的奖励函数，直接驱动模型优化功能正确性。

3. **通过两阶段有监督微调建立基线能力。**本文采用有监督微调（SFT）分别训练两个阶段的模型：对多模态大语言模型进行微调，使其从时序图图像中提取信号名称、电平序列及时钟关系并输出为结构化表示；对纯文本大语言模型进行微调，使其将结构化表示转换为语法正确的 HDL 代码。两阶段的解耦设计降低了单一模型的学习难度，缩短了每个模型需要处理的序列长度，从而降低显存需求，并使每个阶段都能独立优化。
4. **探索基于仿真奖励的强化学习优化。**SFT 的词元（Token）级损失函数不直接优化生成代码的功能行为。本文将所提出的评价指标作为奖励函数，采用群体相对策略优化（GRPO）算法 [24]，以仿真对拍的 F1 分数作为连续值奖励信号对模型进行强化学习优化，形成“生成-仿真-评分-更新”的闭环，为在 SFT 基线之上继续提升代码的功能正确性提供了可行的优化路径。

实验结果表明，本文所提出的方法在数字电路时序图解析任务上显著超越开源基线模型，并逼近前沿闭源模型的性能水平，验证了从时序图图像到结构化表示、再到 HDL 代码这一自动化流水线的可行性与有效性，为智能 EDA 工具的发展开辟了新的路径。

1.3 论文组织结构

本文其余部分的组织结构如下：

第二章介绍问题背景，包括大语言模型与多模态大语言模型的架构原理、数字电路时序图的 WaveJSON 表示格式、评价指标与有监督微调及强化学习的核心方法、面向硬件设计与时序图分析的已有工作，以及本文的问题定义。

第三章详细阐述本文提出的时序图自动化解析框架，涵盖数据合成链路的优化设计、多层级质量预选与防泄漏数据划分、专用评价指标的构建方法、两阶段有监督微调策略以及基于强化学习的进一步优化。

第四章展示实验结果与分析，包括数据集统计分析、评价指标有效性验证、两阶段模型的定量评估、生成代码的错误模式分析以及强化学习阶段的实验结果。

第五章总结全文工作，讨论现有方法的局限性，并展望未来的研究方向。

第 2 章 问题背景

本章介绍本文研究所涉及的关键技术背景与相关工作。首先介绍关键技术基础，包括大语言模型与多模态大语言模型的基本架构、时序图的 WaveJSON 结构化表示格式、结构化输出的评价指标，以及有监督微调与强化学习方法；随后综述面向硬件设计与时序图分析的已有工作，明确本文的研究定位；最后给出本文的问题定义，将时序图解析建模为两阶段生成任务。

2.1 关键技术基础

2.1.1 大语言模型

本文的时序图解析框架依赖大语言模型的序列建模与代码生成能力。Transformer 架构由 Vaswani 等人于 2017 年提出 [4]，其核心思想是用自注意力（Self-Attention）机制替代传统的循环或卷积结构来建模序列中的长距离依赖关系。给定输入序列的查询（Query）、键（Key）和值（Value）矩阵 Q 、 K 、 V ，自注意力的计算公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2.1)$$

其中 d_k 为键向量的维度。多头注意力（Multi-Head Attention）将 Q 、 K 、 V 分别投影至 h 个不同的子空间后并行计算注意力，再将结果拼接投影回原始维度，从而捕获不同位置与不同表示子空间中的信息。Transformer 采用编码器-解码器结构，每一层由多头注意力与前馈网络交替组成，并辅以残差连接与层归一化，在机器翻译等序列到序列任务上取得了突破性效果。

在 Transformer 的基础上，大语言模型（Large Language Model, LLM）采用仅解码器（Decoder-only）架构，通过在大规模文本语料上进行自回归预训练来学习语言的统计规律与语义知识。GPT-2[25] 首次展示了语言模型作为无监督多任务学习器的潜力；GPT-3[26] 通过将参数规模扩展至 1750 亿证明了少样本上下文学习（In-Context Learning）能力的涌现；GPT-4[27] 进一步提升了多步推理与代码生成能力；GPT-5[16] 采用多模型协同架构实现了推理能力的跨越式提升，其后续迭代 GPT-5.5[5] 在代码生成与智能体任务上持续强化。在开源领域，Meta 的 LLaMA 系列经历了快速迭代：LLaMA 2[28] 以 70 亿至 700 亿参数的稠密架构率先缩小了开源与闭源模型的差距；LLaMA 3[29] 将稠密模型规模扩展至 4050 亿参数，在多语

言与代码推理任务上进一步逼近前沿闭源模型的表现；LLaMA 4[6] 转向混合专家（Mixture of Experts, MoE）架构并原生支持多模态输入，以 170 亿激活参数实现了与更大规模稠密模型相当的性能。阿里巴巴的 Qwen3[7] 提供了从 6 亿到 2350 亿参数的完整模型谱系，同时支持稠密与 MoE 两种架构，在代码生成与结构化输出任务上表现突出，且同步提供了对应的多模态版本 Qwen3-VL[20]，使得文本与视觉任务可在统一的模型家族内完成，这一特性使其成为本文两阶段框架的理想基座模型。深度求索的 DeepSeek-V4[8] 以 1.6 万亿总参数、490 亿激活参数的 MoE 架构在代码与数学推理任务上展现了优异性能；智谱的 GLM-4.5[30] 以 3550 亿总参数的 MoE 架构在智能体、推理与代码任务上取得了领先表现；OpenAI 开源的 GPT-OSS-120B[31] 则以 1170 亿总参数的 MoE 架构提供了高质量的代码生成能力。这些模型在自然语言理解、代码生成、逻辑推理等任务上已展现出强大的通用能力，为后续面向特定领域的微调应用奠定了基础。

2.1.2 多模态大语言模型

本文的核心任务，即从时序图图像中提取结构化信息，本质上是一个视觉理解问题，需要模型同时具备图像感知与语言生成能力。多模态大语言模型（Multimodal Large Language Model, MLLM）正是为此类跨模态任务而设计的，它将视觉感知能力与语言理解能力相融合。当前主流 MLLM 的架构通常由三个模块组成：视觉编码器（Vision Encoder）、投影层（Projector）和语言模型（Language Model）。

视觉编码器负责将输入图像转换为一系列视觉特征向量。视觉 Transformer（Vision Transformer, ViT）[32] 是当前最主流的视觉编码器架构，它将图像划分为固定大小的图块（Patch），将每个图块线性映射为嵌入向量后送入 Transformer 编码器进行处理。CLIP[33] 通过在约 4 亿对图像-文本数据上进行对比学习预训练，使视觉编码器学会了与语言语义对齐的图像表示，其预训练的 ViT 编码器被 LLaVA、InternVL 等多个开源 MLLM 直接采用作为视觉感知的基础组件。

投影层的作用是将视觉编码器输出的特征映射到语言模型的词嵌入空间中，弥合视觉与语言两种模态之间的表示差异。不同模型采用了不同复杂度的投影策略：LLaVA[18] 使用简单的线性层或两层多层感知机（Multi-Layer Perceptron, MLP）作为投影器，以极低的参数开销实现了有效的模态对齐；InternVL[34] 则采用了动态高分辨率策略与更深的 MLP 投影，以适应不同分辨率的视觉输入。

语言模型作为 MLLM 的核心推理引擎，接收视觉特征与文本指令的拼接序列，以自回归方式生成文本输出。在闭源模型中，GPT-4o[35] 率先实现了原生多模态统一架构，GPT-5[16] 进一步将多模态推理能力提升至新的水平；Google 的 Gemini 系列从 Gemini 1.0[36] 演进至 Gemini 2.5[17]，后者结合可配置的思维链推理，在

视觉理解与代码生成基准上均达到了前沿水平。在开源领域，LLaVA[18] 率先提出了视觉指令微调（Visual Instruction Tuning）范式，通过构造指令跟随格式的视觉问答数据对 MLLM 进行端到端微调，使轻量级开源模型在视觉理解任务上取得了接近闭源模型的性能。InternVL[34] 通过扩大视觉编码器的规模并改进视觉-语言对齐策略，缩小了开源与闭源模型之间的差距；其最新版本 InternVL3.5[19] 通过原生多模态预训练与混合偏好优化策略，在多模态推理基准上接近前沿闭源模型。Meta 的 LLaMA 4[6] 则将多模态能力内建于模型架构中，是较早原生支持图像与文本联合理解的开源 MoE 模型之一。

本文的视觉理解模块基于 Qwen-VL 系列模型。Qwen2.5-VL[37] 在架构设计上引入了原生动态分辨率（Native Dynamic Resolution）机制，允许模型按照图像的原始宽高比进行处理，而非强制裁剪为固定尺寸，从而避免了信息损失。同时，该模型采用多模态旋转位置编码（Multimodal Rotary Position Embedding, MRoPE）来统一处理文本与视觉标记的位置信息。Qwen3-VL[20] 在此基础上引入了三项关键改进：

1. **DeepStack 融合机制**：不同于传统方法仅使用视觉编码器最后一层的输出，DeepStack 将多个中间层的视觉特征分别注入到语言模型的不同层中，显著增强了模型对细粒度视觉信息（如小字体文本、密集图表）的感知能力。
2. **交错 MRoPE（Interleaved-MRoPE）**：将旋转位置编码的维度在低频与高频段间交错分配，而非按时间、水平、垂直方向简单分组，改善了长序列场景下的位置编码稳定性。
3. **灵活的模型规模**：Qwen3-VL 同时提供稠密模型（2B/4B/8B/32B 参数）与 MoE 模型（30B-A3B/235B-A22B 参数）两类变体，可根据推理延迟与性能需求灵活选择。

凭借上述改进，Qwen3-VL 在多模态理解基准上取得了领先的性能表现，是本文选用的视觉理解基础模型。

2.1.3 WaveJSON 表示格式

数字电路时序图以时钟周期为横轴、信号电平为纵轴，直观地展示了数字系统中各信号随时间变化的行为。它是硬件设计与验证中不可或缺的视觉规范：设计者通过时序图定义模块的输入输出关系、状态转移条件以及信号间的时序约束。在工程实践中，时序图广泛出现于通信协议数据手册（如 SPI、I2C、UART）、总线协议规范（如 AMBA AXI[1]、AHB[2]、APB[3]）以及模块级的设计文档中。为实现时序图的自动化解析，需要一种能够将时序图的视觉信息完整编码为文本序列的结构化表示格式，使其既可由模型生成，又可被程序化地解析与比较。

WaveDrom[38] 是一个被广泛使用的开源数字时序图渲染引擎，它接受基于 JSON 的描述格式 WaveJSON 作为输入，将其渲染为 SVG 矢量图。WaveJSON 的核心结构是一个包含 signal 数组的 JSON 对象，每个信号由 name（信号名称）和 wave（波形字符串）两个基本字段定义。波形字符串中的每个字符代表一个时钟周期内的信号状态，主要字符及其含义如表 2.1 所示。例如，以下 WaveJSON 描述了一个包含时钟、使能、数据总线和应答信号的简单时序图：

```
{ "signal": [
  { "name": "clk",    "wave": "p....." },
  { "name": "en",     "wave": "01...0." },
  { "name": "data",   "wave": "x.=.x.",
    "data": ["D0", "D1"] },
  { "name": "ack",    "wave": "0..1.0." }
]
```

其中 clk 为上升沿时钟信号，字符 p 定义时钟模式，后续点号使其持续运行。en 信号在第二个周期拉高、在第六个周期恢复低电平。data 为数据总线信号，每个等号字符标识一个新的数据值，其标签 D0 与 D1 由 data 字段指定。ack 信号在第四个周期拉高表示应答、在第六个周期恢复低电平。此外，WaveJSON 还支持信号分组、相位偏移、边沿标注等高级特性，能够描述任意复杂度的数字时序图。

表 2.1 WaveJSON 波形字符及其含义

字符	含义
0 / 1	低电平 / 高电平
p / n	上升沿 / 下降沿时钟信号
P / N	带箭头标记的上升沿 / 下降沿时钟信号
=	数据总线信号（默认颜色），标签由 data 字段指定
2-9	不同颜色的数据总线信号，标签由 data 字段指定
x	未定义或无关状态
.	保持前一周期状态不变

在本文的数据生成流水线中，WaveJSON 既是时序图的结构化中间表示格式，也是渲染图像的输入。上述 WaveJSON 示例经 WaveDrom 渲染后得到的时序图如图 2.1 所示。具体流程为：首先利用 Icarus Verilog[39] 对 Verilog 模块及其测试平台进行仿真，生成标准的 VCD（Value Change Dump）波形文件；随后通过开源工具

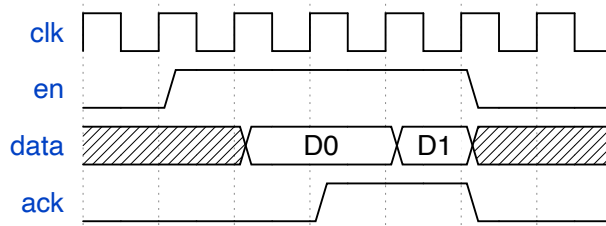


图 2.1 上述 WaveJSON 示例渲染得到的时序图

vcd2wavedrom[40] 将 VCD 文件转换为 WaveJSON 格式；最终由 wavedrom-cli[41] 将 WaveJSON 渲染为时序图图像。这一流程实现了从 HDL 源代码到时序图图像及其结构化表示的全自动生成，为训练数据的大规模构建奠定了基础。

2.1.4 评价指标

本文的时序图解析任务要求模型生成结构化的 WaveJSON 表示或可执行的 Verilog 代码，需要能够准确衡量这类结构化输出质量的评价指标。在自然语言处理领域，BLEU[42] 与 ROUGE[43] 是两种被广泛使用的自动化评价指标。BLEU 基于 n -gram 精确率衡量生成文本与参考文本的匹配程度，ROUGE 则侧重于召回率。此外，Levenshtein 编辑距离 [44] 通过计算将一个字符串转换为另一个字符串所需的最少单字符编辑操作（插入、删除、替换）次数来度量两个序列之间的差异，由此导出的 Levenshtein 比率将编辑距离归一化为 $[0, 1]$ 区间的相似度分数，也常被用于文本匹配与序列对比。然而，这类基于文本表面形式的指标在评估结构化输出时存在固有局限：它们对元素的排列顺序敏感，却对语义关键信息的错误缺乏区分度。当评估对象从自然语言文本转变为 WaveJSON 等结构化表示时，需要设计能够捕捉结构语义正确性的专用评价指标。这类指标不仅服务于模型效果的离线评估，还可以作为奖励信号引入强化学习框架，为模型的训练优化提供定量反馈。

2.1.5 有监督微调

有监督微调（Supervised Fine-Tuning, SFT）是使预训练模型适应特定任务的基本手段。SFT 利用任务的标注数据对模型进行进一步训练，通过向模型展示输入-输出对的示例，使其学会目标任务的输入输出映射。在 MLLM 的应用场景中，SFT 通常采用视觉指令微调范式 [18]：将图像与文本指令拼接作为输入，以期望的文本回答作为监督信号，端到端地优化模型的语言生成参数。

由于大语言模型的参数量通常达到数十亿甚至千亿级别，对全部参数进行微调的计算开销极为庞大。LoRA（Low-Rank Adaptation）[45] 通过在预训练权重矩阵旁引入可训练的低秩分解矩阵来大幅减少可训练参数量。具体而言，对于预训练权

重矩阵 $W_0 \in \mathbb{R}^{d \times k}$, LoRA 将增量参数化为 $\Delta W = BA$, 其中 $B \in \mathbb{R}^{d \times r}$ 、 $A \in \mathbb{R}^{r \times k}$, 秩 $r \ll \min(d, k)$ 。训练过程中仅更新 A 和 B , 而 W_0 保持冻结, 使可训练参数量降低数个数量级, 同时保持与全参数微调相当的性能。

2.1.6 强化学习

SFT 使模型掌握了基本的任务能力, 但其训练目标仅为最大化标注数据的似然, 无法直接优化任务的最终评价指标。强化学习可弥补这一不足。基于可验证奖励的强化学习 (Reinforcement Learning with Verifiable Rewards, RLVR) 利用可自动验证的客观指标作为奖励信号, 而非依赖人工偏好数据训练奖励模型。RLVR 特别适用于具有明确正确性判据的任务, 例如数学求解中答案的正确与否、代码生成中编译与测试的通过与否。在本文中, 功能仿真的通过与否即为一种天然的可验证奖励信号。

群体相对策略优化 (Group Relative Policy Optimization, GRPO) [24] 是 RLVR 框架下的一种高效策略优化算法。与近端策略优化 (Proximal Policy Optimization, PPO) [46] 需要额外训练价值网络来估计优势函数不同, GRPO 通过对同一输入采样一组候选输出, 以组内各输出的奖励相对排名来估计优势值。设对输入 x 采样 G 个输出 $\{y_1, \dots, y_G\}$, 每个输出获得奖励 r_i , 则第 i 个输出的优势估计为:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})} \quad (2.2)$$

这一设计消除了训练价值网络的额外开销, 同时通过组内相对比较提供了稳定的优化信号。GRPO 的优化目标结合 KL 散度正则化项, 在提升生成质量的同时防止策略偏离预训练模型过远。

2.2 相关工作

2.2.1 大语言模型在硬件设计中的应用

利用大语言模型自动生成硬件描述语言代码是近年来 EDA 领域的热门研究方向。在评测基准方面, Liu 等人 [9] 提出的 VerilogEval 包含了从 HDLBits 题库中收集的 Verilog 代码补全任务, 是该领域最具代表性的评测基准之一。在模型研发方面, Thakur 等人 [10,11] 系统评估了通用 LLM 在 Verilog 生成任务上的表现, 并通过在 Verilog 语料上微调 CodeGen 模型得到了 VeriGen, 证明了领域微调的有效性。NVIDIA 团队的 ChipNeMo [12] 通过对 LLaMA 进行芯片设计领域的持续预训练与微调, 在工程助手、代码生成等任务上大幅超越了通用模型。此外, Blocklove 等人 [14] 探索了基于 LLM 的对话式硬件设计, Fu 等人 [13] 将 LLM 应用于 AI 加

速器的设计自动化。

在多模态输入方面, Chang 等人 [23] 利用电路框图、状态转移图等视觉输入辅助 Verilog 代码生成, 在 GPT-4 系列模型上将测试通过率从 46.88% 提升至 71.81%, 证明了视觉信息对硬件代码生成的辅助价值, 但该工作并未关注时序图的解析与理解。上述工作表明, LLM 已具备理解硬件语义并生成功能正确代码的初步能力, 但将视觉化的时序规范纳入自动化流程的研究仍处于空白。

2.2.2 时序图自动化分析

时序图的形式化处理有着较长的研究历史。Fisler[47] 较早地对时序图进行了形式化定义, 提出了基于时间区间的算法验证方法。Maler 与 Nickovic[48] 提出了针对连续信号时序属性的监控方法, Bartocci 等人 [49] 则对信号时序逻辑 (Signal Temporal Logic, STL) 规范的挖掘方法进行了全面综述。这些工作为时序行为的形式化描述与验证奠定了理论基础, 但其输入通常为结构化的信号数据而非图像。

在从时序图图像中自动提取信息方面, He 等人 [21] 提出的 TD-Magic 是较早的尝试, 它通过光学字符识别 (Optical Character Recognition, OCR)、边缘检测等计算机视觉技术从时序图图像中提取严格偏序 (Strict Partial Order, SPO) 关系作为形式化规范。但该方法基于规则的多模块设计限制了其处理复杂场景的能力, 且仅适用于包含显式偏序关系的少数时序图。同一团队后续提出的 TD-Interpreter[22] 采用了截然不同的技术路线, 通过对开源 MLLM LLaVA 进行微调, 构建了一个面向时序图的视觉问答系统。该工作开发了包含描述型与推理型两类合成数据的生成流程, 使微调后的模型在时序图理解任务上大幅超越了未经微调的 GPT-4o, 证明了通过领域特定数据微调 MLLM 以理解时序图的可行性。

本文的工作与上述方法存在本质区别。TD-Magic 局限于提取偏序关系这一单一类型的形式化规范。TD-Interpreter 虽然证明了领域微调的可行性, 但存在以下五方面不足: 一是任务定位局限于交互式问答, 其输出为自然语言回答, 不涉及结构化信息的完整提取或 HDL 代码的生成, 无法直接服务于设计自动化流程; 二是训练数据中描述型问答仅涉及信号数量、电平值等简单事实查询, 而推理型问答依赖人工设计问题模板与标注答案, 难以大规模扩展; 三是其数据合成流程采用基于随机激励的测试平台生成方案, 仿真产生的时序图往往缺乏有意义的功能行为模式, 限制了训练数据的质量与多样性; 四是评价体系沿用 BLEU、ROUGE 等通用文本相似度指标, 这类指标对元素排列顺序敏感但对语义关键错误缺乏区分度, 无法准确衡量时序图结构化表示的语义正确性; 五是仅采用有监督微调进行训练, 未探索利用可验证奖励信号进行强化学习优化, 限制了模型输出的功能正确性。而本文将时序图解析建模为从图像到结构化表示、再到 HDL 代码的生成式

流水线，训练数据可通过仿真工具链全自动生成，旨在实现对时序图完整信息的提取与硬件行为的逆向工程，在任务深度、数据可扩展性与实用价值上均超越了现有工作。

2.3 问题定义

本文将数字电路时序图的自动化解析形式化为以下两阶段生成任务。

第一阶段：时序图图像到结构化表示。给定一张时序图图像 I ，模型需要生成对应的 WaveJSON 结构化表示 J 。 J 包含一个信号数组，每个信号由名称 (name) 和波形字符串 (wave) 组成，多位数据信号还包含数值标签 (data)。该阶段要求模型从像素级的视觉输入中准确识别所有信号的名称、电平序列与时钟关系，并将其编码为结构化的文本表示。

第二阶段：结构化表示到 HDL 代码。给定第一阶段输出的 WaveJSON 表示 J ，模型需要生成能够产生相同时序行为的可综合 Verilog 模块 V 。该阶段要求模型从信号的时序变化中推理出底层的硬件逻辑（包括组合逻辑关系、有限状态机的状态转移、计数器与移位寄存器等时序电路结构），并将其转化为语法正确且功能等价的 RTL 代码。

上述定义中，第一阶段输出的结构化表示 J 除了作为第二阶段的输入外，其结构化特性使输出可被程序化地解析、比较与验证，为自动化评估与后续处理提供了客观基础。

第 3 章 方 法

本章详细阐述本文提出的数字电路时序图自动解析框架。首先给出框架的整体架构与数据合成链路的总体流程；随后分别从测试平台生成策略与数据质量预选两个维度阐述训练数据的构建方法；接着介绍防泄漏数据划分策略；然后提出面向时序图结构化表示的专用评价指标；之后描述基于有监督微调的两阶段模型训练方法；最后介绍基于评价指标的强化学习优化方法。

3.1 框架概览

针对第 2.3 节定义的两阶段任务，本文设计了如图 3.1 所示的解析框架。

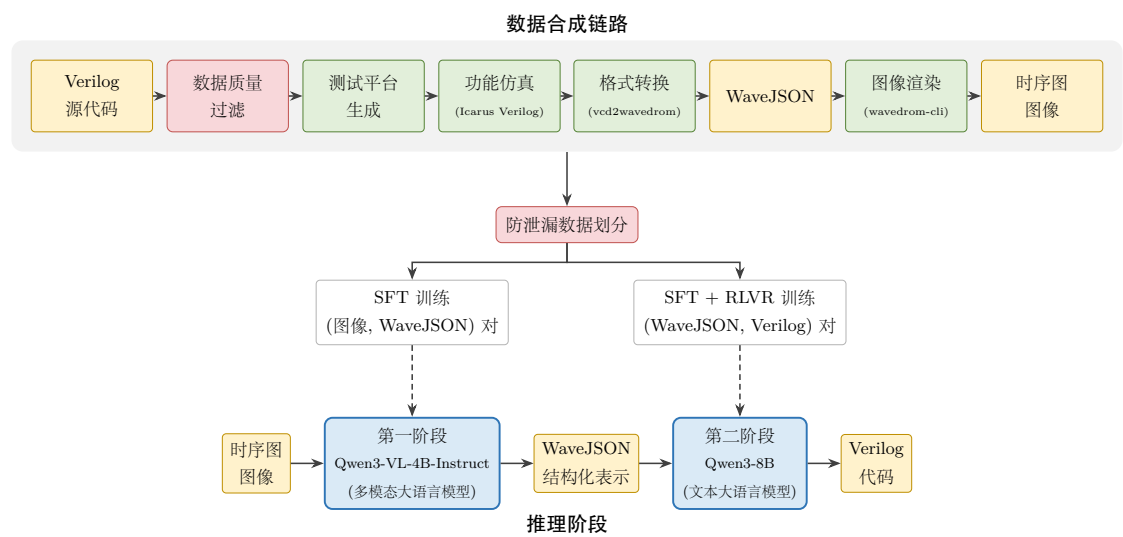


图 3.1 时序图自动解析框架概览

第一阶段采用多模态大语言模型 Qwen3-VL-4B-Instruct 实现图像到 WaveJSON 的转换，以时序图的 PNG 图像与文本提取指令作为输入，自回归地生成结构化表示。**第二阶段**采用纯文本大语言模型 Qwen3-8B 实现 WaveJSON 到 Verilog 代码的转换，以第一阶段的输出与生成指令作为输入，输出可综合的 Verilog 模块。

两阶段采用不同模型架构的考虑如下：第一阶段的核心挑战是跨模态视觉理解，需要多模态模型的图像感知能力；第二阶段的核心挑战是逻辑推理与代码生成，纯文本模型在此类任务上更具优势。同时，结构化中间表示使得两个阶段可以独立训练与优化，降低了端到端学习的难度。此外，两阶段解耦显著缩短了每个模型需要处理的序列长度——端到端方案需要模型同时容纳高分辨率图像词元与完整的 Verilog 代码输出，对上下文窗口与显存的需求极高；而解耦后每个阶段

仅需处理任务链中的一段输入输出，降低了对硬件资源的要求，使得在有限的计算资源上训练成为可能。

整个框架的训练依赖大规模的配对数据：时序图图像与 WaveJSON 的配对用于第一阶段，WaveJSON 与 Verilog 代码的配对用于第二阶段。高质量训练数据的匮乏是时序图解析任务面临的核心障碍。与自然图像或文档图像不同，数字电路时序图缺少现成的大规模标注数据集。TD-Interpreter[22] 率先提出了基于仿真的时序图数据合成思路，本文沿用这一基本思路，以公开数据集 Verilog_GitHub[11] 作为 Verilog 源代码的来源。该数据集从 GitHub 公开仓库中收集了大量 Verilog 源文件，为大规模数据合成提供了原始素材。具体而言，该链路以单个可综合 Verilog 模块的源代码为输入，经过以下四个步骤产出训练所需的三元组数据（时序图图像、WaveJSON 结构化表示、Verilog 源代码）：

1. **测试平台生成。**为待测模块自动生成 Verilog 测试平台（Testbench），包含时钟驱动、输入激励序列与 VCD 波形转储语句。
2. **功能仿真。**使用 Icarus Verilog[39] 对模块源代码与测试平台进行联合编译与仿真，生成 VCD（Value Change Dump）波形文件。
3. **格式转换。**使用开源工具 vcd2wavedrom[40] 将 VCD 文件转换为 WaveJSON 格式的结构化表示。
4. **图像渲染。**使用 wavedrom-cli[41] 将 WaveJSON 渲染为 SVG 矢量图，再导出为 PNG 位图作为模型的视觉输入。

在上述流程中，步骤 2 至 4 是确定性的工具链操作，合成数据的质量取决于两个关键因素：步骤 1 中测试平台的激励质量，以及对候选模块的质量预选。本文分别在这两个维度上进行了系统性的改进，将在接下来的两节中详细介绍。

3.2 测试平台生成策略

测试平台的激励质量直接决定了仿真产生的波形是否包含有意义的信号行为模式。激励不足或缺乏功能语义的测试平台会导致波形退化为无意义的常量或噪声，严重影响训练数据的有效性。本文探索了两种测试平台生成方案：基于随机激励的基线方案与基于大语言模型的优化方案。

3.2.1 基于随机激励的基线方案

基线方案沿用 TD-Interpreter[22] 的测试平台生成思路，使用 edautils 工具包中的 gentbvlog 工具 [50] 为 Verilog 模块自动生成测试平台。gentbvlog 是一个基于 Java 的测试平台生成器，其工作流程为：解析 Verilog 模块的端口声明，识别时钟

信号（按名称匹配 `clk` 或 `clock`），为时钟信号生成固定频率的周期性驱动，为其余输入端口生成伪随机激励序列。

该方案的优势在于完全自动化且适用面广——只要 Verilog 模块语法正确，就能生成可仿真的测试平台。仿真运行时，时钟信号的最大仿真周期数被限制为 20 个，以控制生成时序图的长度。

然而，随机激励的固有局限也很明显：由于激励序列不具备任何功能语义，仿真产生的波形往往缺少有意义的信号行为模式。例如，一个有限状态机模块在随机输入下可能始终停留在复位状态，使得输出信号在整个仿真过程中保持常量不变。这类“退化”样本对模型训练的贡献有限，随机激励方案产生的低质量样本比例较高，限制了训练数据的有效规模。

3.2.2 基于大语言模型的优化方案

为克服随机激励缺乏功能语义的问题，本文引入大语言模型来生成具有工程意义的测试平台。该方案以随机激励方案的数据合成链路为基础，复用其仿真、格式转换与图像渲染等基础设施，仅替换测试平台生成环节。本文使用 GPT-OSS-120B 模型 [31]（一个 1170 亿总参数的 MoE 架构开源大语言模型）通过 OpenAI 兼容 API 进行测试平台生成，其核心思想是：将 Verilog 模块的源代码提供给 LLM，令其分析模块功能后生成针对性的激励序列，使仿真产生的波形能够充分展示模块的各种工作状态。

本文设计了一套结构化的提示模板，要求 LLM 在生成测试平台时遵循以下关键规则：

1. **DUT 例化规范。**测试平台模块命名为 `testbench`，将待测模块例化为 `dut`（小写），端口连接必须与模块声明完全匹配。
2. **初始化要求。**在 `initial` 块中将所有输入端口驱动至已知值，避免出现未定义状态（X 态）。
3. **时钟约束。**对含时钟的设计，使用 `forever #10 clk = ~clk` 生成 20ns 周期的时钟信号，仿真时间固定为 400ns（20 个完整时钟周期）；对不含时钟的纯组合逻辑设计，仿真时间同样为 400ns，以 10ns 为步长产生 40 个时间步，每个时间步施加一组新的输入激励。
4. **激励分布约束。**要求激励序列在整个仿真时间窗口内均匀分布，至少在 80% 的时钟周期内包含有意义的信号活动，避免激励集中在前几个周期而后续长时间空闲。
5. **时序对齐。**所有信号变化必须发生在时钟沿或半周期整数倍时刻，禁止使用非半周期整数倍的延时（如 `#1`、`#5`），以确保 VCD 到 WaveJSON 转换后波

形的时间分辨率统一。

6. **VCD 转储。**测试平台中必须包含指定路径的 `$dumpfile` 与 `$dumpvars` 调用。
7. **仿真终止。**使用独立的 `initial` 块在固定时刻调用 `$finish`，不在激励块中提前终止仿真。

LLM 的输出通过结构化标签 (`<testbench_verilog>` 与 `<skip_reason>`) 进行解析：若模型判断模块无法生成有效测试平台（如功能不明确），则返回跳过原因；否则返回完整的测试平台代码。

生成的测试平台经过与基线方案相同的后续流程（Icarus Verilog 仿真、VCD 转换、WaveJSON 生成与图像渲染），最终产出训练数据三元组。相比随机激励方案，LLM 生成的测试平台能够针对模块的具体功能设计有目的的激励序列：对有限状态机模块遍历各状态转移路径，对计数器模块测试边界溢出行为，对算术单元模块覆盖典型运算模式。这使得仿真产生的时序图包含更丰富、更具区分度的信号行为模式，从而为模型训练提供更高质量的监督信号。此外，具有功能覆盖性的测试平台在评估阶段同样发挥重要作用：如第 3.5.4 小节所述，第二阶段通过仿真对拍评估生成代码的功能正确性，激励序列越能覆盖模块的关键行为模式，对拍结果就越能有效区分功能正确与功能错误的生成代码。

3.3 数据质量预选

除测试平台的激励质量外，对候选模块的质量预选是控制数据合成成本与保障数据有效性的另一关键维度。由于 LLM 测试平台生成的 API 调用成本较高，需要通过低成本的预选机制筛除不适合进入 LLM 管线的模块，例如无法独立仿真的模块、仿真产出尺寸畸形图像的模块等。本文设计了贯穿数据合成全流程的多层级预选机制，按作用阶段分为两个层次。

3.3.1 Verilog 源码预筛

在进入仿真流程之前，对 Verilog 源文件进行以下检查，排除无法独立仿真的模块：

- 排除含有 ``include` 指令的文件，因为其依赖外部文件无法独立仿真。
- 排除使用 `$readmemh` 系统调用的文件，因为缺少外部数据文件。
- 排除标注了黑盒 (blackbox) 属性的模块。
- 排除不含任何行为描述 (always 块、assign 连续赋值或门级原语实例化) 的空壳模块。

- 使用 Icarus Verilog 进行语法检查，排除语法不正确的文件。
 - 排除包含多个 module 定义的文件，确保每个数据样本对应单一的待测模块。
 - 排除模块名匹配工艺库标准单元命名模式（如以 sky130、altera、xilinx 等为前缀）的模块，因为这些是工艺库原语而非可综合 RTL 设计。
- 该预筛同时适用于随机激励方案与 LLM 优化方案，是所有后续处理的前提条件。

3.3.2 基于随机管线的仿真产出质量预选

通过源码预筛的 Verilog 模块进入随机激励方案的数据合成管线。编译或仿真失败的模块不会产出 VCD 文件，自动排除在数据集之外。对成功产出时序图图像与 WaveJSON 的样本，进一步进行多维度的质量筛选：

- **图像尺寸约束。**设置像素总数的上下界，排除过小（信息不足）或过大（超出模型处理能力）的图像。
- **宽高比约束。**限制图像宽高比在合理范围内，排除极端形状的图像。
- **信号数量约束。**限制 WaveJSON 中的信号数量，排除信号过少或过多的样本。

该筛选同时为 LLM 优化方案提供候选模块的预选。由于 LLM API 调用成本较高，本文利用随机管线的产出作为低成本的探针，先排除具有极端尺寸或形状的样本，仅将通过全部筛选的模块送入 LLM 管线，从而显著降低 API 调用开销。

3.4 防泄漏数据划分

在将过滤后的数据划分为训练集与测试集时，必须防止训练集与测试集之间的信息泄漏——即避免高度相似的样本同时出现在两个集合中，导致评估结果虚高。本文采用基于并查集（Disjoint Set Union, DSU）的防泄漏划分策略，通过多维度的相似性检测将相似样本归为同一等价类，确保同一等价类的所有样本被完整地分配到同一集合中。

相似性检测包含以下两个维度：

1. **WaveJSON 指纹匹配。**对每个样本计算其 WaveJSON 的规范化指纹：首先对所有信号名进行规范化（统一小写、去除空格、规范化总线索引格式），然后将每个信号的规范化名称与波形字符串组成二元组，按信号名排序后序列化为确定性字符串作为指纹。指纹相同的样本被归入同一等价类。
2. **信号名模糊匹配。**在波形拓扑多重集（将波形字符串抽象为高/低/数据/保持等拓扑符号的多重集合）相同的样本之间，计算规范化信号名序列的相似度。

相似度超过阈值（默认 0.88）的样本对被合并。这一机制能够检测出仅因端口重命名而产生的变体样本。

两个维度的检测结果通过并查集进行传递性合并，最终按等价类为单位进行训练/测试集的整组划分。

3.5 评价指标

在介绍模型训练方法之前，本文首先定义面向时序图结构化表示的专用评价指标。该指标既用于量化评估模型的解析质量，也将在后续强化学习阶段作为奖励函数发挥核心作用。

对于第一阶段（图像到 WaveJSON），该指标直接将模型预测的 WaveJSON 与标准答案 WaveJSON 进行结构化比较。对于第二阶段（WaveJSON 到 Verilog 代码），由于模型输出不再是 WaveJSON，本文通过功能仿真将生成代码的行为转换回 WaveJSON 后再进行比较，具体方法将在第 3.5.4 小节介绍。

传统的文本相似度指标（如 BLEU[42]、ROUGE[43]）无法有效衡量 WaveJSON 解析结果的正确性。例如，两个 WaveJSON 在信号顺序不同但内容完全一致时，文本相似度会显著降低；而一个波形字符串中单个字符的错误可能导致后续所有时钟周期的语义偏移，但文本相似度的下降却不与语义损失成正比。因此，本文设计了面向 WaveJSON 结构特性的专用评价指标。

3.5.1 信号匹配

评价的第一步是建立预测结果与标准答案之间的信号对应关系。本文采用两阶段匹配策略：

精确匹配阶段。对预测与标准答案中的信号名进行规范化处理（统一大小写、去除空格、规范化总线索引格式），然后进行一对一的精确匹配。

模糊匹配阶段。对未在精确匹配阶段配对的信号，使用 Levenshtein 编辑距离 [44] 计算信号名之间的相似度。采用贪心策略，优先配对相似度最高的信号对，相似度低于阈值（默认 0.7）的不予配对。

模糊匹配的引入是为了应对信号名的轻微变体（如 `data_out` 与 `dataout`），同时信号名匹配的相似度将作为后续波形评分的权重因子。

3.5.2 波形评分

对每对匹配的信号，将波形字符串展开为逐周期的词元序列：保持字符（.）被替换为前一周期的实际状态值，数据字符（=）被替换为 `data` 数组中对应的

标签文本。展开后的两条词元序列使用 Levenshtein 比率计算相似度，其定义为：

$$\text{ratio}(s_1, s_2) = \frac{|s_1| + |s_2| - d(s_1, s_2)}{|s_1| + |s_2|} \quad (3.1)$$

其中 $d(s_1, s_2)$ 为两序列间的编辑距离。

此外，针对 WaveDrom 时钟信号的表示特性进行了等价性处理：时钟信号既可以用专用字符（p 表示上升沿、n 表示下降沿）表示，也可以用 0/1 交替表示。评分时会枚举两种表示方式的所有初相组合，取最高分作为最终得分，避免因表示风格差异而产生不必要的扣分。将上述流程所得的单信号对得分记为 $\text{WaveScore}(g, p)$ ，其中 g 与 p 分别为标准答案信号与预测信号。

3.5.3 综合评分

对每个样本，将所有匹配信号对的波形得分（乘以信号名匹配的相似度权重）进行汇总，计算精确率（Precision）、召回率（Recall）与 F1 分数：

$$\text{Precision} = \frac{\sum_{(g,p,s) \in M} s \cdot \text{WaveScore}(g, p)}{|P|} \quad (3.2)$$

$$\text{Recall} = \frac{\sum_{(g,p,s) \in M} s \cdot \text{WaveScore}(g, p)}{|G|} \quad (3.3)$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.4)$$

其中 M 为匹配信号对集合， (g, p, s) 分别为标准答案信号名、预测信号名与信号名匹配相似度， $\text{WaveScore}(g, p)$ 为对应的波形得分， $|G|$ 与 $|P|$ 分别为标准答案与预测结果中的信号总数。

精确率衡量预测结果中有多大比例是正确的，召回率衡量标准答案中有多大比例被正确识别，F1 分数是两者的调和平均。该指标从信号名的匹配准确性与波形序列的时序一致性两个层面对解析结果进行量化评估。

3.5.4 基于仿真对拍的代码评估

第二阶段的任务是从 WaveJSON 生成 Verilog 代码，其输出不再是 WaveJSON 结构，无法直接应用上述指标。本文采用基于功能仿真的对拍验证方法，将生成代码的行为转换回 WaveJSON 后与标准答案进行指标对比。该方法不直接比较代码文本的相似度，而是比较两者在相同激励下的功能行为是否一致。这一设计的合理性在于，数字电路设计中功能等价的模块可以有截然不同的实现方式，直接比较文本会将合理的实现差异误判为错误。具体流程如下：

首先从模型输出中提取 Verilog 代码段。由于 WaveJSON 不包含模块名称、参数声明、端口方向等接口元信息，生成代码的接口规格可能与测试平台不匹配。为

使评估聚焦于模型的逻辑推理能力，本文在编译前自动执行接口对齐：将生成模块名重写为测试平台期望的 DUT 类型名；从测试平台的参数覆写中提取缺失参数注入生成模块；若编译器报告端口方向冲突则迭代修复。这些操作仅修改接口声明而不改变内部逻辑。

随后，将对齐后的代码与该样本对应的原始测试平台通过 Icarus Verilog 联合编译并运行仿真，生成 VCD 波形文件，再转换为 WaveJSON 后与标准答案进行上述指标的对比评分。每个样本的测试平台在数据合成阶段已生成并通过验证，其激励序列经 LLM 针对性设计，覆盖了模块的主要工作模式与边界条件，保证了评估条件的确定性与可复现性。经接口对齐后仍无法编译的样本（如代码存在语法错误或结构残缺）视为生成失败，F1 计为零。

3.6 有监督微调

3.6.1 第一阶段：时序图图像到 WaveJSON

第一阶段使用 Qwen3-VL-4B-Instruct[20] 作为基础模型，通过 LoRA[45] 进行有监督微调。训练数据为时序图 PNG 图像与 WaveJSON 的配对，以 ShareGPT 多轮对话格式 [51] 组织：用户消息包含图像标记与提取指令，助手消息为对应的 WaveJSON 文本。

训练的关键超参数设置如下：

- LoRA 秩设为 32，应用于模型的全部线性层。
- 冻结视觉编码器（Vision Tower）与多模态投影层的参数，仅训练语言模型部分的 LoRA 适配器。这一策略基于以下考虑：预训练的视觉编码器已具备充分的图像特征提取能力，时序图理解的瓶颈在于语言模型对视觉特征的解读方式，而非视觉特征本身的质量。
- 图像最大像素数限制为 1,048,576（约 1024×1024 ），序列截断长度为 8192 个词元（Token）。
- 总批大小为 64，学习率为 1×10^{-4} ，使用恒定学习率调度，训练 15 轮（Epoch）。

3.6.2 第二阶段：WaveJSON 到 Verilog 代码

第二阶段使用 Qwen3-8B[7] 作为基础模型，同样通过 LoRA 进行有监督微调。训练数据为 WaveJSON 与 Verilog 模块源代码的配对：用户消息包含 WaveJSON 文本与生成指令，助手消息为对应的 Verilog 代码（已去除注释）。

训练的关键超参数设置如下：

- LoRA 秩设为 64（高于第一阶段），因为代码生成任务的输出空间更大，需

要更强的适配能力。应用于模型的全部线性层。

- 采用 FSDP (Fully Sharded Data Parallel) [52] 进行分布式训练, 使用 `full_shard` 策略, 将模型参数、梯度与优化器状态分片存储于多块 GPU 上以降低单卡显存占用。
- 序列截断长度为 8192 个词元 (Token)。
- 总批大小为 64, 学习率为 2×10^{-5} , 使用恒定学习率调度, 训练 15 轮 (Epoch)。

两个阶段的训练均使用 LlamaFactory 框架 [53] 完成, 采用 BF16 混合精度训练以降低显存占用。

3.7 基于强化学习的优化

有监督微调使模型学会了从 WaveJSON 到 Verilog 代码的基本映射, 但生成代码的功能正确性仍有提升空间。为此, 本文探索引入基于可验证奖励的强化学习 (Reinforcement Learning with Verifiable Rewards, RLVR) 进行进一步优化。

3.7.1 奖励函数设计

RLVR 的关键是设计可自动验证的奖励信号。对于第二阶段的 WaveJSON 到 Verilog 生成任务, 本文利用功能仿真构建连续值奖励函数, 其计算流程如下:

1. **代码提取。**从模型的生成文本中解析 Verilog 代码段, 优先提取代码围栏 (Code Fence) 内的内容, 其次匹配 `module...endmodule` 结构。
2. **接口对齐与编译。**对提取的代码执行第 3.5.4 小节所述的三层接口对齐 (模块名重写、参数注入、端口方向修复), 随后与测试平台联合编译并运行仿真, 生成 VCD 波形文件。
3. **波形对比评分。**将 VCD 文件转换为 WaveJSON 后, 使用第 3.5 节描述的评价指标与标准 WaveJSON 进行对比, 计算 F1 分数。

最终奖励值直接取波形对比的 F1 分数, 为 $[0, 1]$ 区间内的连续值; 若上述任一步骤失败 (如编译错误、仿真超时), 则奖励为零。连续值的奖励信号相比二值判定提供了更细粒度的优化梯度, 使模型能够从部分正确的生成结果中获得正反馈。该奖励函数完全客观且可自动化执行, 不依赖人工标注或主观判断, 仅依赖确定性的仿真工具链与结构化的评分算法。

3.7.2 GRPO 优化

本文采用群体相对策略优化 (GRPO) [24] 算法进行强化学习训练, 基于 `verl` 框架 [54] 实现。对每个训练样本, 模型采样一组候选 Verilog 代码, 每份代码独立

进行功能仿真与评分，所得 F1 分数作为该候选的奖励值。GRPO 通过组内奖励的相对排名估计优势函数，消除了训练价值网络的额外开销。

训练数据与 SFT 阶段相同，将 WaveJSON 与对应的测试平台路径、参考 Verilog 代码等元信息打包为 Parquet 格式。强化学习阶段在 SFT 微调后的检查点上继续训练，关键超参数设置如下：

- 沿用 SFT 阶段的 LoRA 配置：秩为 64，缩放因子 α 为 128，应用于全部线性层。
- 对每个训练样本采样 4 个候选输出用于组内比较。
- 训练批大小为 32，小批次大小为 16，学习率为 3×10^{-6} 。
- 禁用 KL 散度惩罚项。由于 SFT 已将模型锚定在领域数据分布上，且训练步数有限，策略偏移风险较低，故取消 KL 约束以允许模型在仿真奖励的引导下充分探索。
- 使用 vLLM[55] 作为推理引擎进行候选代码的批量生成，采用张量并行加速。
- 共训练 230 步。

整个流程形成“生成—仿真—评分—更新”的闭环优化，逐步提升生成代码的功能正确性。

第 4 章 实验结果

本章展示实验结果与分析。首先对构建的数据集进行多维度统计分析，验证其质量与多样性；随后通过对照实验验证本文评价指标的有效性；接着分别报告两阶段模型的定量评估结果与错误模式分析；最后报告强化学习优化及与通用大模型对比的实验结果。

4.1 数据集统计分析

本节对经第 3 章所述流程构建的最终数据集进行多维度统计分析，从数据规模、信号复杂度、代码长度、图像尺寸与模块功能类型等方面验证数据集的质量与多样性，为后续模型训练与评估提供量化依据。

4.1.1 数据规模

本文的数据源为公开数据集 Verilog_GitHub[11]，该数据集从 GitHub 公开仓库中收集了 108,971 个 Verilog 源文件。经过第 3 章所述的多层级筛选流程后，最终保留 5,847 个有效模块用于数据合成。

从 108,971 个原始文件到最终 5,847 个有效样本，整体保留率约为 5.4%。大量文件在筛选过程中被排除，主要原因包括：Icarus Verilog 编译错误（语法不合规或使用了不支持的语言特性）、仿真运行失败（如死循环导致超时、信号未正确初始化等）、依赖外部文件（``include` 指令、`$readmemh` 系统调用）导致无法独立仿真、包含多模块定义、属于工艺库标准单元而非可综合 RTL 设计，以及仿真产生的图像尺寸或信号数量超出合理范围等。

4.1.2 信号数量与代码长度分布

每个样本的信号数量反映了对应电路模块的端口复杂度，Verilog 源代码的长度则反映了模块的逻辑复杂度。图 4.1 展示了数据集中样本的信号数量分布。信号数量主要集中在 5 至 10 个之间，占比约 67%，中位数为 8 个信号，覆盖了从简单门电路（3–4 个端口）到中等复杂度模块（13–15 个端口）的广泛范围，为模型训练提供了不同复杂度层次的样本。

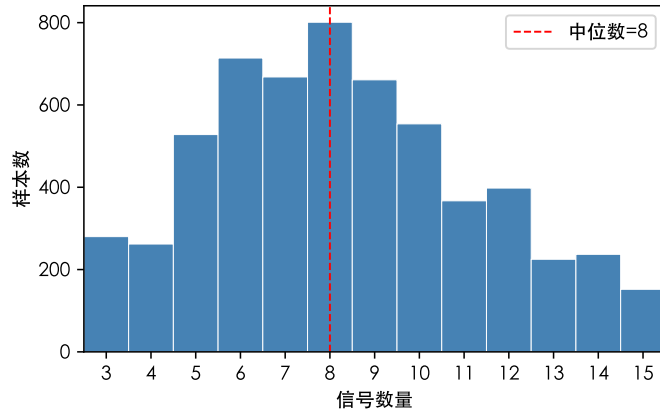


图 4.1 样本信号数量分布

4.1.3 Verilog 代码长度分布

图 4.2展示了去除注释与空行后各样本的有效代码行数与词元数分布。代码行数的中位数为 61 行，均值为 89 行，主要集中在 21 至 120 行之间（约 75%）。词元数（基于 Qwen3-8B 分词器）的中位数为 558，均值为 956，主要集中在 200 至 1,200 之间（约 73%）。两个维度均呈现右偏分布，少数大型模块（如总线协议控制器、多级流水线运算单元）的代码长度显著高于平均水平。整体而言，数据集中的模块以中等规模为主，兼顾了训练效率与逻辑复杂度的多样性。

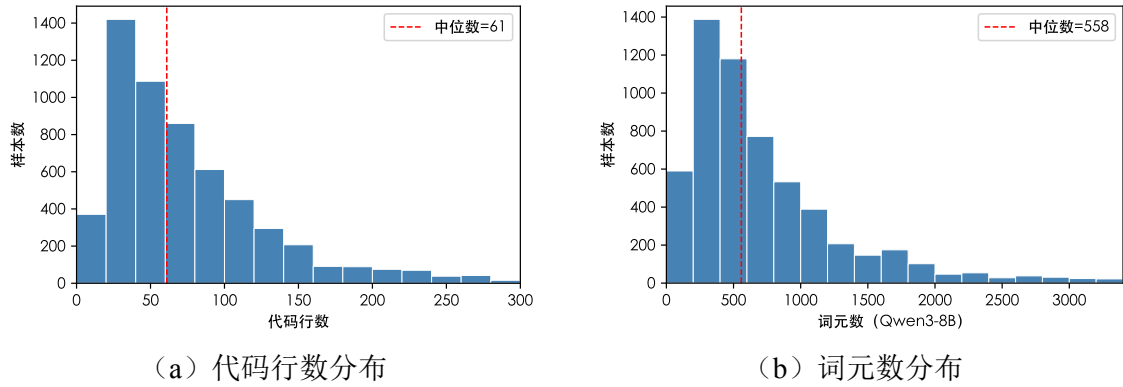


图 4.2 Verilog 代码长度分布

4.1.4 时序图图像尺寸分布

时序图图像的尺寸特征决定了视觉模型的输入规格要求。图 4.3展示了数据集中时序图图像的宽高比与像素总数分布。

时序图图像普遍呈极端横向形状：宽高比中位数为 9.56，主要分布在 4 至 20 之间（占 98%），这是由时序图以时钟周期为横轴、信号堆叠为纵轴的固有结构决定的。图像宽度中位数为 1,780 像素，高度中位数为 240 像素，像素总数中位数为 510K。约 69% 的图像像素总数在 300K 至 1,000K 之间，少数包含大量信号或长时

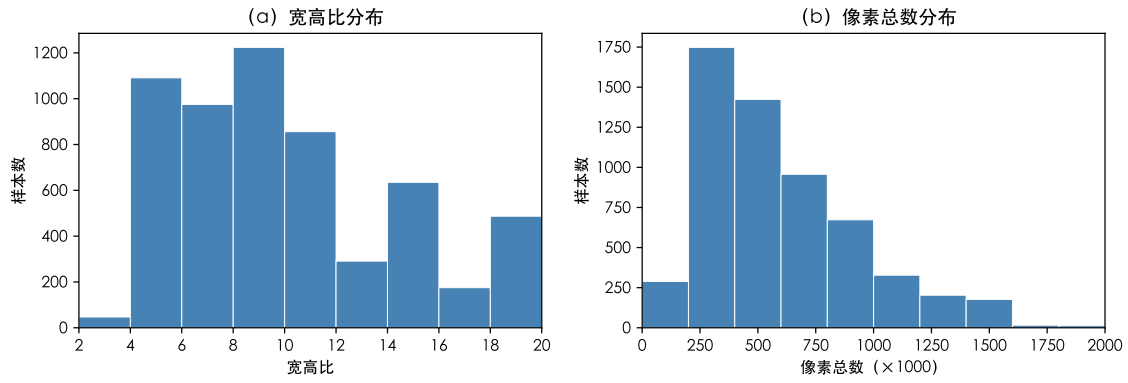


图 4.3 时序图图像的宽高比与像素总数分布

间跨度的样本超过 1,000K 像素。这一尺寸分布为第一阶段模型的图像输入参数设置（最大像素数 1,048,576）提供了依据。

4.1.5 数据划分

经过第 3 章所述的防泄漏划分策略，5,847 个样本被划分为训练集 5,730 个与测试集 117 个。两个阶段的训练与评估均使用相同的划分方案。

4.1.6 模块功能多样性

为评估数据集覆盖的电路功能类型，本文基于模块名称的自动化分段匹配对 5,847 个模块进行了功能分类。具体方法为：首先从测试平台文件中提取被测模块（DUT）的 Verilog 模块名；然后将模块名按下划线与驼峰命名边界拆分为词段序列（例如 `axi_protocol_converter_v2_1_b2s_simple_fifo` 拆分为 `[axi, protocol, converter, ..., simple, fifo]`）；接着将每个词段与预定义的关键词-类别映射表进行匹配，该映射表的关键词来源于对全部 2,362 个不同模块名的词段频率统计，选取出现频率最高的功能性词段（如 `fifo`、`fsm`、`counter`、`mux` 等）作为类别关键词。

关键词分为两个层级：功能关键词（描述电路功能，如 `fifo`、`counter`）与上下文关键词（描述总线接口，如 `axi`、`spi`、`uart`）。分类规则为：当模块名中恰好出现一个功能关键词时，归入对应类别；仅出现上下文关键词时，归入通信控制器；出现多个不同类别的功能关键词时，为避免歧义，保守地归入“其他”类别。分类结果如表 4.1 所示。

基于模块名的分类方法具有高精度——每个分类结果均可直接从模块名验证，且分类规则不涉及任何启发式歧义消解。为评估分类的可靠性，本文将模块名分类结果与 LLM 生成测试平台时输出的模块功能描述文本进行交叉验证：对每个被分类的模块，检查其 LLM 分析文本中是否包含与分类类别相关的功能关键词。

表 4.1 数据集模块功能分类统计

功能类别	模块数	占比
通信控制器	540	9.2%
FIFO	350	6.0%
时钟与同步	258	4.4%
状态机	197	3.4%
多路选择器	170	2.9%
算术逻辑单元	166	2.8%
存储器	161	2.8%
定时器	129	2.2%
计数器	125	2.1%
GPIO 与外设	76	1.3%
编解码器	50	0.9%
CRC 与校验	47	0.8%
寄存器堆	36	0.6%
移位寄存器	35	0.6%
其他	3,507	60.0%

交叉验证结果显示，95.6% 的分类结果被 LLM 分析文本所确认，其中 6 个类别的确认率达到 100%，表明基于模块名的分类具有较高的可靠性。未被确认的 4.4% 主要来自 LLM 分析文本未显式提及功能关键词的情况（如仅列出端口表而未描述模块功能），而非实际的分类错误。

约 40% 的模块被成功分类。通信控制器占比最高（9.2%），主要来源于 AXI、SPI、UART、PCIe 等总线接口与串行通信接口的子模块；FIFO（6.0%）与时钟同步电路（4.4%）紧随其后，反映了数据缓冲与跨时钟域同步在数字设计中的普遍性。算术逻辑单元（2.8%）包含整数运算器与浮点运算子模块，存储器（2.8%）涵盖了单端口与双端口 RAM、ROM、缓存等。状态机、多路选择器、定时器、计数器等常见功能模块均有分布。

“其他”类别（60.0%）的构成较为多样，主要包括：项目专用名称或缩写无法自动识别功能的模块（如 `csrbrg`、`ablk` 等），含有通用功能词但语义过于宽泛的模块（如各类 `controller`、`control` 模块），FPGA IP 核中的数据通路适配子模块（如总线宽度转换器 `DecWidthConverter`、协议适配器 `MM_to_ST_Adapter` 等），以及少量多关键词冲突而保守归入“其他”的模块。

整体而言，已分类模块涵盖了从基础逻辑单元到中等复杂度时序电路的广泛功能类型，表明数据集具备良好的功能多样性。

4.1.7 时序图样本示例与激励方案对比

本节通过具体样本展示两种测试平台生成方案产生的时序图差异，以直观说明激励质量对训练数据的影响。

状态机对比。图 4.4 展示了一个多状态 FSM 模块 (FsmExample) 在两种激励下的时序图。该模块以两个输入信号 a、b 驱动状态转移，通过 3 位输出 dout 指示当前状态。随机激励方案中，gentbvlog 正确识别了时钟信号并生成了周期性驱动，复位信号 rst_n 也恰好经历了正确的置位-释放序列，状态机在复位后正常运行。然而，输入信号 a 与 b 的随机组合缺乏功能覆盖性——在整个仿真过程中仅产生了 4 种可能输入组合中的 3 种，缺少同时为高的组合 (a=1, b=1)，导致 dout 只在状态 1 与状态 2 之间切换，状态 3 从未被触发。LLM 激励方案在分析模块的状态转移逻辑后，设计了覆盖性的输入序列：先完成规范的复位初始化，随后系统地组合 a 与 b 的不同取值以覆盖全部转移路径，输出 dout 在 20 个时钟周期内循环经过全部状态值 (1 → 2 → 1 → 3 → 1 → 2 → 3 → 2 → ...)，完整展示了状态机的转移行为。

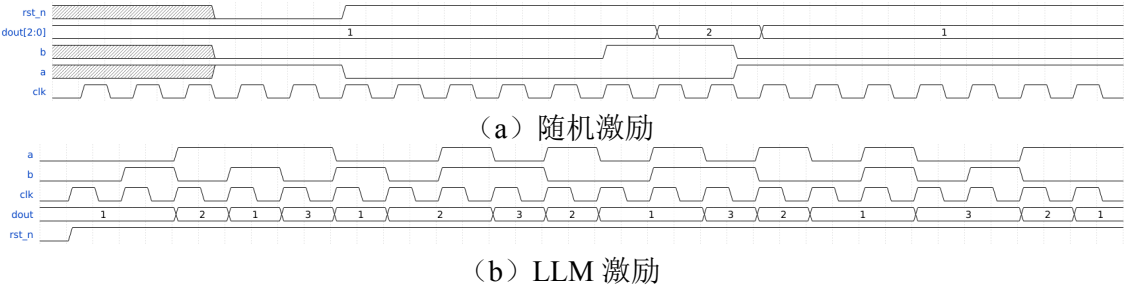


图 4.4 同一状态机模块在两种激励方案下的时序图对比

FIFO 对比。图 4.5 展示了同一 8 位 FIFO 模块 (8 个信号) 在两种激励下的时序图。两种方案均使用了正确的周期性时钟，差异完全来自激励策略。随机激励方案中，复位信号长时间保持有效，使 FIFO 大部分时间处于复位状态；写使能与读使能的跳变缺乏协调，数据输入端口呈现无意义的随机十六进制值，rd_data 在整个仿真过程中仅输出一个有效值——FIFO 的核心功能未能被充分激活。LLM 激励方案在分析模块功能后设计了结构化的操作序列：先短暂断言复位信号完成初始化，随后进入入队阶段依次写入具有工程辨识度的测试向量 (0xAA、0xBB、0xCC 至 0x22 共 8 个值)，当 FIFO 写满时 full 标志正确置位；接着切换为出队操作，rd_data 按先入先出顺序从 0xAA 开始依次输出全部写入数据，empty

标志在 FIFO 清空时正确置位。full 与 empty 的状态翻转清晰可见，完整展示了 FIFO 的缓冲与流控行为。

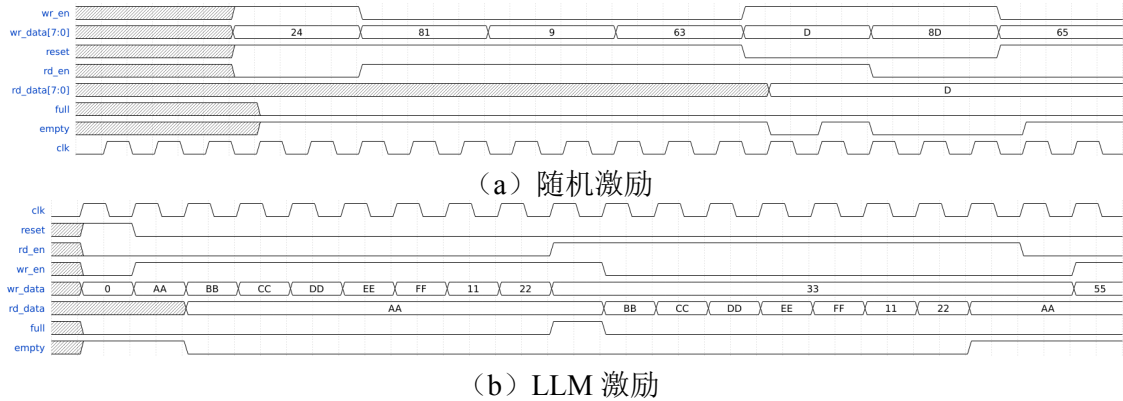


图 4.5 同一 FIFO 模块在两种激励方案下的时序图对比

上述两组对比揭示了随机激励方案的核心局限：缺乏功能语义的激励序列使得模块的关键行为模式（状态遍历、标志位翻转等）无法在仿真中被充分激活，产生的时序图对模型训练贡献有限。LLM 激励方案通过理解模块功能语义生成有目的的激励，使时序图能够充分展示模块的各种工作状态，为模型提供更具区分度的训练样本。同时，高覆盖率的激励序列也使得第二阶段的仿真对拍评估更加可靠——随机激励下大量输出信号保持常量，即使生成代码存在功能错误也可能碰巧通过对拍；而 LLM 激励充分激活了模块的各种工作状态，使对拍结果能够更有效地检测生成代码的功能缺陷。

图 4.6 展示了数据集中不同功能类别模块的时序图样本，体现了数据集在信号类型与波形模式上的多样性。超前进位加法器模块以纯组合逻辑为主，无时钟信号，输入 a、b 与进位 cin 驱动求和输出 s 与进位输出 cout，波形呈现密集的数据标签跳变；UART 控制器模块展示了串行通信接口的典型交互模式，包含写数据 wr、发送请求 trx_req、串行输出 trx 与应答 trx_ack 等握手信号；SRAM 存储器模块展示了典型的片上存储读写时序：芯片使能 CEN 与写使能 WEN 协同控制写入操作，测试数据 D（0xAA、0x55、0xFF 等）被依次写入不同地址；读取阶段输出使能 OEN 拉低，数据输出 Q 从高阻态转为有效数据。七个信号均呈现清晰的状态变化，完整展示了存储器的使能控制、地址寻址与数据读写时序关系。

4.2 评价指标有效性分析

第 3.5 节提出了面向时序图结构化表示的专用评价指标。为验证该指标相比传统文本相似度指标的有效性，本节通过一组对照实验进行对比分析。对照基线选用

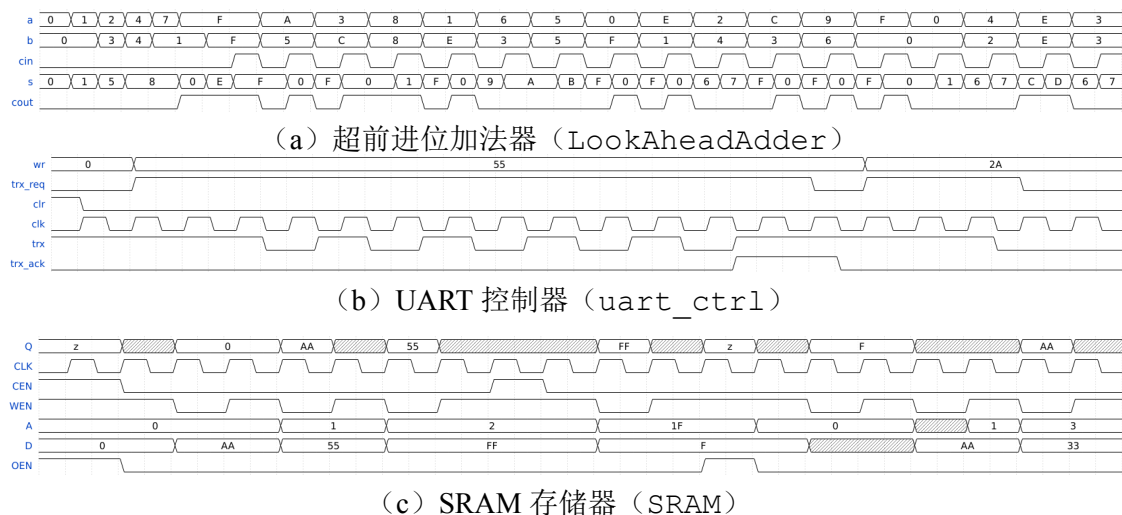


图 4.6 数据集中不同功能类别的时序图样本

两种广泛使用的文本相似度指标：Levenshtein 比率 [44]（将标准答案与预测结果的 WaveJSON 序列化文本进行字符级编辑距离比较）与 BLEU[42]（将 WaveJSON 序列化文本按词元分割后计算 n -gram 精确率）。

本文选取五个具有代表性的评估场景，分别对应时序图解析中常见的差异类型，涵盖了文本相似度“虚低”（语义一致但文本不同）与“虚高”（语义不同但文本相似）两类典型失效模式。各场景的评分对比结果如表 4.2 所示。

表 4.2 传统文本相似度指标与本文评价指标的对比

编号	场景	Levenshtein	BLEU	本文指标		
				P	R	F1
1	信号顺序置换	0.912	0.952	1.000	1.000	1.000
2	保持符号等价	0.939	0.845	1.000	1.000	1.000
3a	少量数据标签错误（1/4）	0.991	0.928	0.875	0.875	0.875
3b	大量数据标签错误（4/4）	0.966	0.777	0.500	0.500	0.500
4	信号缺失（4 缺 1）	0.897	0.801	1.000	0.750	0.857

场景 1：信号顺序置换。如图 4.7 所示，预测结果包含与标准答案完全相同的三个信号（时钟、输入、输出），仅信号在 WaveJSON 数组中的排列顺序不同，渲染后时序图中信号的纵向排列位置发生了变化，但每个信号的波形内容完全一致。由于信号排列顺序不改变任何时序语义，因此应获得满分。Levenshtein 比率因数组元素位置变化产生了 8.8% 的扣分，BLEU 虽受影响较小（0.952）但仍未给出满分，而本文指标通过基于信号名的匹配策略正确给出 1.0 的满分。

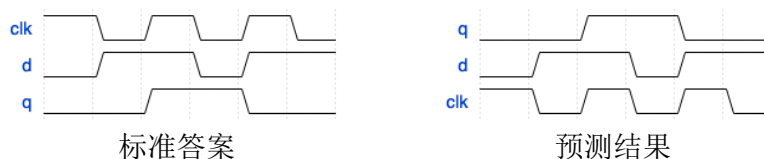


图 4.7 场景 1：信号顺序置换（语义一致，仅纵向排列不同）

场景 2：保持符号等价。如图 4.8所示，标准答案与预测结果描述的信号行为完全相同，差异仅在于 WaveJSON 的书写方式：标准答案使用保持符号（.）表示信号在后续时钟周期中延续前一周期的电平（如 0..1.. 表示低电平持续三拍后高电平持续三拍），预测结果则逐周期显式写出电平值（如 000111）。两种写法的语义完全等价，但文本差异显著。Levenshtein 比率产生了 6.1% 的扣分，BLEU 的扣分更为严重（0.845），而本文指标在评分前将保持符号展开为实际电平值后进行比较，正确给出满分。

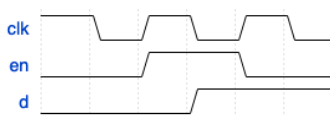


图 4.8 场景 2：保持符号等价（0..1.. vs 000111，渲染后视觉一致）

场景 3：数据标签错误。如图 4.9所示，此场景展示指标对语义错误严重程度的区分能力。标准答案包含一个 4 拍计数器信号，数据标签依次为 0、1、2、3。场景 3a 中仅最后一个标签错误（3→4），场景 3b 中全部四个标签均错误。从渲染的时序图可以直观看出两种错误的严重程度差异显著。然而 Levenshtein 比率在两种情况下的得分分别为 0.991 与 0.966，差异仅为 0.025；BLEU 分别为 0.928 与 0.777，差异为 0.151，区分能力优于 Levenshtein 但仍不充分。而本文指标的 F1 分数分别为 0.875 与 0.500，差异达到 0.375，准确反映了错误从局部到全局的严重程度变化。

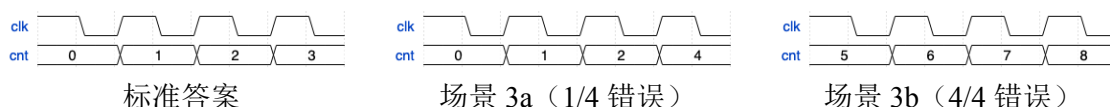


图 4.9 场景 3：数据标签错误（标签 3→4 vs 全部替换）

场景 4：信号缺失。如图 4.10所示，预测结果遗漏了标准答案中四个信号之一（信号 b 缺失）。Levenshtein 比率给出 0.897，BLEU 给出 0.801，均未能充分反映信息缺失的严重性。本文指标的 F1 为 0.857，其中精确率（Precision）为 1.000（预测的信号全部正确），召回率（Recall）为 0.750（标准答案中有 25% 的信号未被识别）。这一精确率与召回率的分解提供了比单一相似度数值更丰富的诊断信息，有助于分析模型的错误模式——是倾向于“少预测”（召回率低）还是“乱预测”（精确率低）。

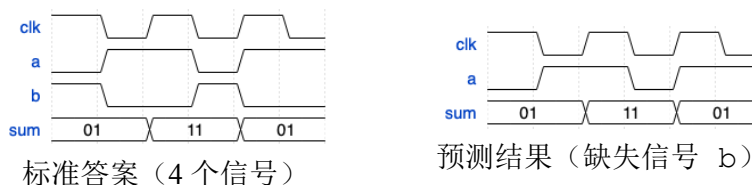


图 4.10 场景 4：信号缺失（信号 b 被遗漏）

综合以上分析，Levenshtein 比率与 BLEU 等传统文本相似度指标存在两类系统性偏差：对语义无关的文本差异（信号顺序、表示风格）产生不必要的惩罚（场景 1、2），同时对语义关键的内容错误（数据标签、信号缺失）缺乏足够的敏感度（场景 3、4）。本文设计的评价指标通过信号级匹配、保持符号展开与精确率-召回率分解，在两个方向上均给出了更符合语义正确性的评分。

4.3 第一阶段有监督微调实验结果

本节报告第一阶段（时序图图像到 WaveJSON）有监督微调模型在测试集上的评估结果。模型基于 Qwen3-VL-4B-Instruct 微调，使用 vLLM 推理引擎在 117 个测试样本上进行推理，将模型预测的 WaveJSON 与标准答案 WaveJSON 通过第 3.5 节所述指标进行比较。

4.3.1 总体结果

为评估微调的效果，本文将微调模型与多个通用多模态大语言模型进行对比，使用相同的测试集与评估流程。基线模型包括微调前的同架构模型 Qwen3-VL-4B-Instruct 以及 Qwen3-VL 系列的大参数量版本，均以零样本方式直接推理。为确保对比的公平性，所有基线模型的提示词中均详细说明了 WaveJSON 的格式规范与输出要求，包括信号数组的结构定义、波形字符的含义以及数据标签的使用方式，使模型在不经微调的情况下也能充分理解任务目标与期望的输出格式。表 4.3 汇总了各模型的对比结果。

表中“解析成功”指模型输出的文本能被 JSON 解析器成功提取为包含 signal 数组的合法 WaveJSON 对象；未能解析的样本 F1 计为零。通用多模态模型在零样本设置下的表现显著低于微调模型。微调前的同架构模型 Qwen3-VL-4B-Instruct 仅有 16 个样本（13.7%）被成功解析，平均 F1 仅为 0.006，表明该模型在未经领域微调时几乎不具备时序图结构解析能力。Qwen3-VL-235B-A22B-Thinking 通过启用思维链推理获得了基线中最高的平均 F1（0.334），但仍远低于本文 4B 参数的微调模型（0.969）。解析成功率方面，通用模型的输出格式遵从性较低，仅 13.7% 至 88% 的样本能被成功解析为合法 WaveJSON，而微调模型达到 99.1%。这

表 4.3 第一阶段不同模型的评估结果对比

模型	参数量	解析成功	平均 F1
Qwen3-VL Plus[20]	未公开	88/117	0.226
Qwen3-VL-235B-A22B-Instruct[20]	235B (A22B)	75/117	0.298
Qwen3-VL-235B-A22B-Thinking[20]	235B (A22B)	103/117	0.334
Qwen3-VL-4B-Instruct[20]	4B	16/117	0.006
Qwen3-VL-4B-Instruct[20] + SFT（本文）	4B	116/117	0.969

一对比表明，时序图的结构化解析是一项高度专业化的任务，即使具备强大视觉理解能力的大规模通用模型也无法通过零样本推理有效完成，领域特定的微调不可或缺。值得注意的是，微调后的 4B 模型性能大幅超越了参数量高出数十倍的通用模型，凸显了高质量领域数据与针对性训练的重要性。

在 117 个测试样本中，116 个样本（99.1%）的模型输出被成功解析为合法的 WaveJSON 结构，仅 1 个样本因输出格式异常导致解析失败，F1 计为零。各样本的 F1 分数分布如图 4.11 所示。

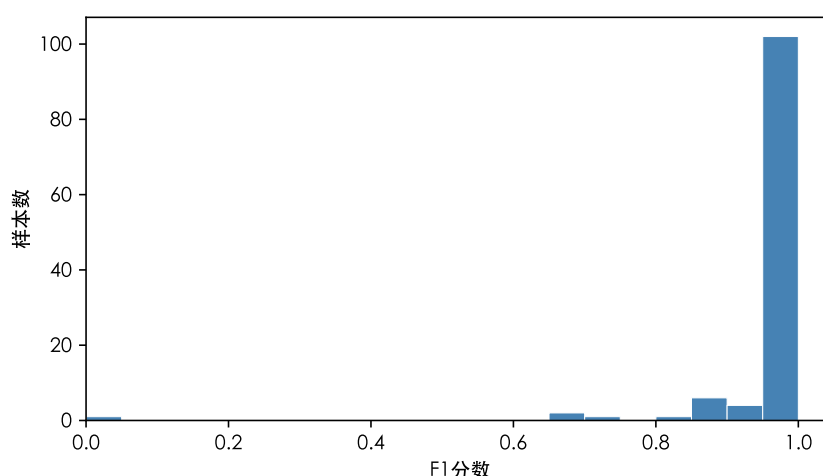


图 4.11 第一阶段测试集 F1 分数分布

分布高度集中于高分区间：73 个样本（62.4%）达到 $F1=1.0$ （完全正确），102 个样本（87.2%）的 $F1 \geq 0.95$ ，106 个样本（90.6%）的 $F1 \geq 0.9$ 。全部 117 个样本的平均 F1 为 0.969，平均精确率与平均召回率均为 0.969。

4.3.2 解析成功样本分析

仅考虑成功解析的 116 个样本，平均 F1 达到 0.977，中位数为 1.000。73 个样本（62.9%）的 F1 为 1.0，表明模型在这些样本上完整且准确地还原了时序图中所

有信号的名称、波形序列与数据标签。F1 低于 0.95 的样本仅有 14 个（12.1%），F1 低于 0.8 的样本仅有 3 个（2.6%），说明模型在绝大多数样本上的解析质量较高。

图 4.12 展示了一个 F1=1.0 的复杂样本，即一个 FIFO 读控制器模块的输入时序图。该模块包含 14 个信号，涵盖时钟与复位信号、7 个数据总线信号（共 41 个数据标签）以及 5 个二值控制信号，波形中包含密集的数据标签跳变与复杂的控制逻辑交互。模型完整且准确地还原了全部信号的名称、波形序列与数据标签值。



图 4.12 第一阶段完全正确的复杂样本：FIFO 读控制器模块（14 个信号，F1=1.0）

4.3.3 错误模式分析

对 F1 低于 0.95 的 14 个样本进行信号级对比分析，可归纳出以下两类主要错误模式。

波形长度偏差。最常见的错误类型是模型预测的波形字符串长度与标准答案不一致，即模型对时序图中的时间步总数估计有误。在 14 个低分样本中，超过半数存在此类问题。

图 4.13 展示了一个典型案例的输入时序图。该样本为一个纯组合逻辑模块（F1=0.650），包含 4 个数据总线信号，不含时钟信号。标准答案的波形字符串长度为 20，而模型预测的波形字符串长度为 40。具体而言，标准答案中每个信号的波形均为连续的数据字符（如 =====），表示每个时间步都有新的数据值；而模型在每两个数据字符之间插入了保持符号（如 =.），将波形长度扩展为两倍。尽管模型正确识别了全部 20 个数据值及其顺序，但时间步数量的翻倍导致逐步对齐时产生系统性偏移，使 F1 显著下降。

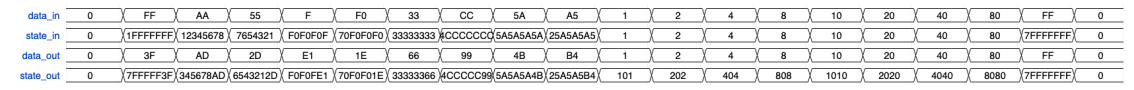


图 4.13 波形长度偏差示例：纯组合逻辑模块的输入时序图（F1=0.650）

如图 4.13 所示，标准答案（波形长度 20）与模型预测（波形长度 40）渲染出的时序图在视觉上几乎完全相同——两者包含相同的数据值序列，区别仅在于 WaveJSON 层面每个数据值是否紧随一个保持符号。正是由于两种波形长度对应的时序图在视觉上无法区分，模型难以从图像中准确判断时间步的总数，从而产生系统性的长度偏差。

数据标签识别错误。部分样本中，模型正确识别了信号名称与波形结构，但数据总线的数值标签存在少量错误。

图 4.14 展示了一个指令译码器模块（F1=0.887）的前 5 个信号，其中模型预测中与标准答案不一致的位置以黄色高亮标注。该模块包含 13 个信号，模型正确识别了全部信号名称与波形的整体结构，但数据标签存在多处错误，且错误分布于多个信号中：寄存器地址输出 DR 有 13 处偏差，源操作数地址 SA 有 19 处偏差，目标操作数地址 SB 有 13 处偏差，立即数输出 IMM 有 25 处偏差。从图中可以观察到，错误集中在波形的后半段——前半段的数据标签基本正确，而从中段开始出现局部错位后，后续标签随之发生连锁偏移。这类错误反映了时序图中小字号数值标签的视觉识别难度，尤其当多位数据标签紧密排列时，字符间的区分度降低，单个标签的识别错误会引发后续标签的连锁错位。



图 4.14 数据标签识别错误示例：指令译码器模块前 5 个信号（F1=0.887）

唯一的解析失败样本是一个包含 12 个信号的 AXI 接口模块。模型实际生成了语义正确的 WaveJSON 内容，但输出的前缀格式异常（在 JSON 前附加了非预期的文本标签），导致 JSON 解析器无法提取有效结构。这属于输出格式遵从性的偶发错误，而非模型视觉理解能力的系统性不足。

整体而言，第一阶段模型在时序图到 WaveJSON 的转换任务上表现优异：信号名称的识别几乎完全正确（所有成功解析的样本均正确识别了全部信号名），波形结构的还原精度高，主要瓶颈在于长时序图的时间步计数精度与密集数据标签的精确识别。

4.4 第二阶段有监督微调实验结果

本节报告第二阶段（WaveJSON 到 Verilog 代码）有监督微调模型在测试集上的评估结果。评估流程如第 3.5.4 小节所述，将模型生成的 Verilog 模块与对应的测试平台联合仿真，通过波形对拍计算 F1 分数。测试集共 117 个样本。

4.4.1 总体结果

为评估微调的效果，本文将微调模型与多个通用大语言模型进行对比，使用相同的测试集与评估流程（含第 3.5.4 小节所述的接口对齐机制）。基线模型包括微调前的同架构模型 Qwen3-8B、多个开源大参数量模型以及闭源商用模型 Claude Opus 4.6[56]，均以零样本方式直接推理。为确保对比的公平性，所有模型的提示词中均详细说明了任务目标、期望的输出格式（完整的 Verilog 模块）以及模块接口的约束条件。表 4.4 汇总了各模型的对比结果。

表 4.4 第二阶段不同模型的评估结果对比

模型	参数量	通过仿真	平均 F1
GLM-4.5-Air[30]	106B (A12B)	44/117	0.316
DeepSeek V4 Flash[8]	284B (A13B)	50/117	0.354
Qwen3-Coder-480B-A35B[57]	480B (A35B)	62/117	0.429
GPT-OSS-120B[31]	117B (A5B)	84/117	0.599
Claude Opus 4.6[56]	未公开	98/117	0.744
Qwen3-8B[7]（零样本）	8B	38/117	0.263
Qwen3-8B + SFT（本文）	8B	84/117	0.648
Qwen3-8B + SFT + GRPO（本文）	8B	98/117	0.692

从表中可以观察到，开源通用模型的零样本 F1 在 0.316 至 0.599 之间，其中 GPT-OSS-120B 表现最优（F1=0.599，通过率 71.8%），尽管其总参数量（117B）低于 DeepSeek V4 Flash（284B）和 Qwen3-Coder（480B），说明模型在代码生成任务上的能力并非仅由参数规模决定，训练数据质量与模型架构设计同样关键。Claude Opus 4.6 闭源模型表现突出，达到 98/117 的通过率与 0.744 的 F1。本文的 SFT 微调模型以仅 8B 参数即达到 F1=0.648，超越了全部开源通用模型，并略优于参数量约 15 倍于己的 GPT-OSS-120B。经 GRPO 优化后，8B 模型在通过率上与 Claude 完全持平（均为 98/117），F1 差距收窄至 0.052（0.692 vs 0.744），表明面向特定任务的数据合成与强化学习训练流水线能够使小规模模型达到接近前沿闭源模型的性能水平。

在 117 个测试样本中，SFT 模型有 84 个样本（71.8%）通过了编译与仿真，33 个样本（28.2%）因语法错误或结构残缺未能产出有效波形，F1 计为零。各样本的 F1 分数分布如图 4.15 所示。

分布呈现明显的两极化特征：33 个样本集中在 F1=0（编译失败），41 个样本

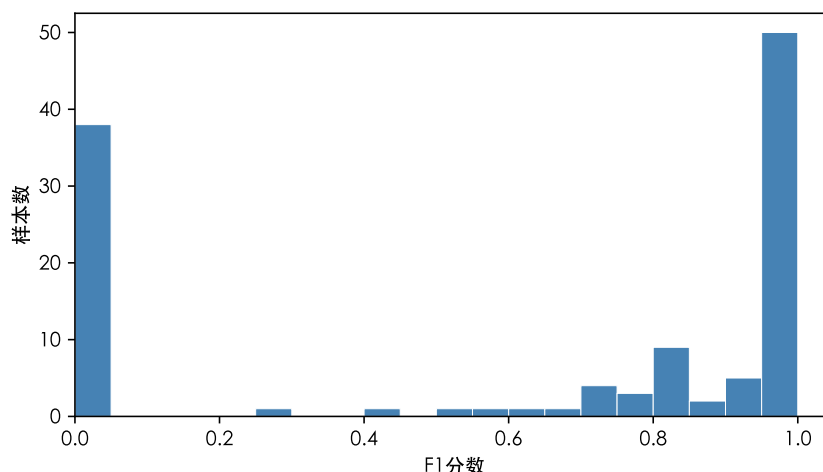


图 4.15 第二阶段测试集 F1 分数分布（SFT 模型）

达到 $F1=1.0$ （完全正确），中间区域的样本相对较少。全部 117 个样本的平均 F1 为 0.648。

4.4.2 仿真成功样本分析

仅考虑通过仿真的 84 个样本，平均 F1 达到 0.902，中位数为 1.000。其中 41 个样本（48.8%）的 F1 为 1.0，表明模型生成的代码在测试平台的全部激励下与标准答案的行为完全一致。 $F1 \geq 0.8$ 的样本共 68 个（81.0%），说明绝大多数成功仿真的样本在功能上高度正确，仅存在局部信号的细微偏差。F1 低于 0.5 的样本仅有 3 个（3.6%），表明模型一旦能生成可编译的代码，其功能正确性普遍较高。

图 4.16 展示了一个 $F1=1.0$ 的复杂样本的输入时序图，对应一个 PCIe QPLL 复位状态机控制器。该模块包含 15 个信号，涵盖时钟锁定检测、DRP 配置、PLL 复位与上电控制等多个子系统的交互逻辑。模型生成了 224 行参数化 Verilog 代码，包含 4 个模块参数、两级异步寄存器同步链、12 状态的有限状态机与 5 个输出组合逻辑赋值，仿真波形与标准答案完全一致。

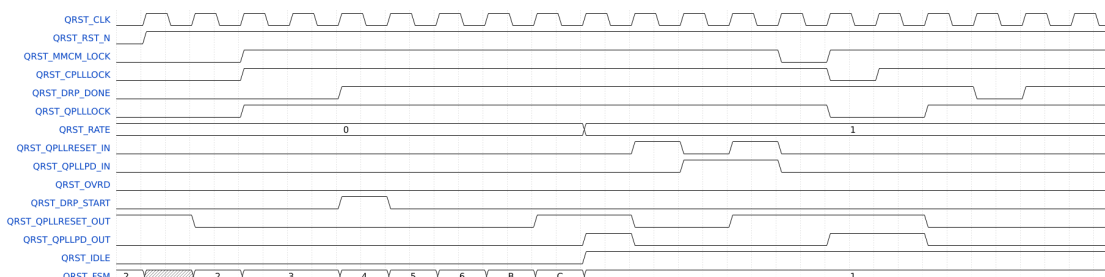


图 4.16 第二阶段完全正确的复杂样本:PCIe QPLL 复位控制器(15 个信号,224 行 Verilog, $F1=1.0$)

4.4.3 仿真失败分析

33 个样本未能通过编译，其中 10 个存在编译期语义错误，6 个因端口赋值类型冲突，5 个因代码截断而缺少 `endmodule`，5 个存在其他语法错误，4 个引用了未定义的标识符，3 个端口声明不匹配。按错误来源可归纳为以下几类。**结构规划不足**（5 个样本）：模型陷入局部模式的机械重复（如生成 1,353 行重复寄存器声明、或尝试暴力枚举 2^{14} 种 `case` 分支）导致输出被词元上限截断，缺少完整的 `endmodule` 结构。**语言规范混淆**（16 个样本）：包括使用 Verilog-2001 不支持的 SystemVerilog 语法（如 `break` 语句）、用 `assign` 驱动 `reg` 类型变量、在同一作用域重复声明标识符等，反映模型未能严格区分不同 Verilog 标准的语法规则与赋值语义。**信号引用错误**（7 个样本）：引用不存在的信号名（如将 `Address` 误写为 `addr`）、端口声明数量或名称与模块定义不一致等。此外，5 个样本存在其他语法错误（如括号不匹配、关键字拼写错误等）。

4.4.4 逻辑实现错误分析

对 43 个通过仿真但 $F1 < 1.0$ 的样本进行信号级分析，可以揭示有监督微调模型最具代表性的错误模式：模型从训练数据中学会了 Verilog 代码的结构框架（端口声明、`always` 块骨架、`case` 语句模板），但在决定功能行为的关键细节上存在系统性偏差。其中 10 个样本存在输出信号退化为常量的现象，以下三个典型案例展示了这类错误的多种表现形式。

组合逻辑译码失效。一个指令译码器模块（ $F1=0.407$ ）的 13 个端口中，输入信号 `INST` 被正确传递，但 12 个译码输出中大部分始终保持零值或未定义态。标准答案中，寄存器地址字段 `DR`、`SA`、`SB` 应从指令中提取不同位段，控制信号 `MB`、`LD`、`MW` 应根据指令类型产生使能脉冲。模型生成的译码器包含了正确的 `case` 语句框架，但译码条件与输出赋值的对应关系存在大量错误，导致绝大多数指令未能命中有效分支。

时序逻辑状态机停滞。一个行扫描控制器（ $F1=0.584$ ）的 15 个信号中，4 个输入信号被正确复现，但 11 个输出中有 5 个始终为高阻态，其余在复位释放后恒为零。标准答案显示该状态机应循环执行行加载序列——`prev_row_load`、`curr_row_load`、`next_row_load` 依次以一个周期的偏移产生使能脉冲。模型生成的代码包含了状态寄存器与转移逻辑的框架，但转移条件不完整，状态机无法离开初始状态。

数值映射偏差。一个相位译码器（ $F1=0.254$ ，通过仿真的样本中最低分）的输入信号被正确复现，但 4 个相位输出的数据标签值与标准答案显著不同。例如，

dqs_phase 的标准标签序列为 16、36、56、77、7、27、48、68，反映相位值随输入地址的查表映射关系，而模型生成的序列为 66、25 的简单交替。模型能够还原信号的跳变时序，但无法准确推断具体的数值映射。

上述三类错误呈现出一致的模式：模型在端口结构、控制流骨架、信号类型等代码框架层面基本正确，但在决定功能行为的关键细节（如译码真值表的具体条目、状态转移的精确条件、数值映射的查找表）上存在系统性偏差。需要指出的是，评估采用相同测试平台的仿真对拍，模型只需生成在该激励下行为一致的任意实现即可获得满分，并不要求与标准答案的代码相同。模型未能找到任何一个功能等价的实现，反映的是模型从行为观测推断逻辑规则的能力不足。

这一错误模式具有重要的方法论启示。有监督微调的词元级损失函数对功能正确性的监督是间接的：它优化的是生成序列与参考代码之间的词元匹配度，而非生成代码的功能行为。上述案例表明，词元级匹配能够引导模型学会代码的结构模式，但不足以保证行为层面的正确性。同时，这类错误产生的部分正确输出（F1 分布在 0.25 至 0.58 之间，而非全零）恰好为基于仿真奖励的强化学习提供了有意义的优化梯度。第 3 章提出的 GRPO 方法正是针对这一瓶颈：以仿真对拍的 F1 分数作为连续值奖励信号，直接在功能正确性层面优化模型。下一节将报告强化学习的实验结果，随后对比分析本文微调模型与通用大模型的能力差异。

4.5 第二阶段强化学习实验结果

本节报告在 SFT 检查点基础上进行 GRPO 强化学习训练后的评估结果。训练共进行 230 步，评估流程与第二阶段 SFT 模型相同。

4.5.1 总体结果

表 4.5 对比了 SFT 模型与 GRPO 模型在测试集上的表现。

表 4.5 GRPO 强化学习优化前后的评估结果对比

模型	通过仿真	平均 F1	仿真成功 F1
Qwen3-8B + SFT	84/117	0.648	0.902
Qwen3-8B + SFT + GRPO	98/117	0.692	0.826

GRPO 训练将仿真通过率从 71.8%（84/117）大幅提升至 83.8%（98/117），净增 14 个样本通过编译与仿真，整体平均 F1 从 0.648 提升至 0.692。这一结果表明，基于仿真奖励的强化学习能够在 SFT 基线之上显著改善生成代码的编译正确性与功能准确性。

图 4.17展示了 GRPO 训练过程中平均 F1 与仿真通过率的变化趋势。两项指标均呈现持续上升趋势，其中仿真通过率从初始的约 54% 稳步提升至 81%，平均 F1 从约 0.50 提升至 0.69。由于训练阶段采用采样温度 $T = 1.0$ 以鼓励探索，曲线相对评估结果存在一定波动。

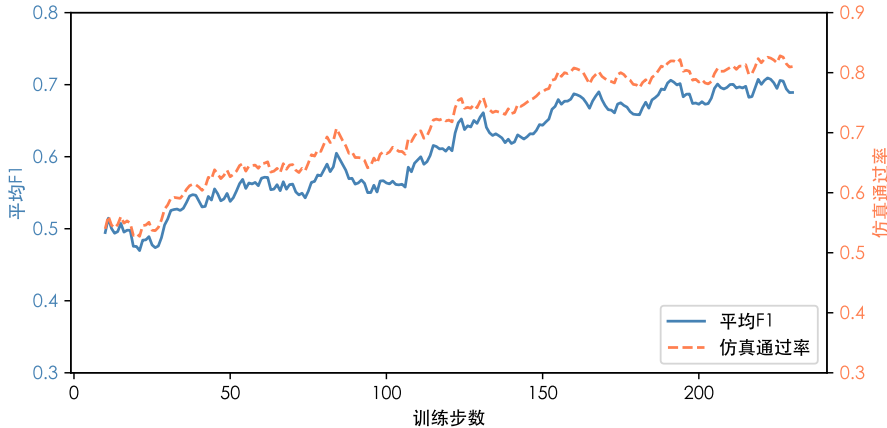


图 4.17 GRPO 训练过程中平均 F1 与仿真通过率的变化曲线

4.5.2 结果分析

对比 SFT 与 GRPO 的结果，可以观察到以下特点：

编译通过率的显著提升。GRPO 模型的仿真通过数从 84 个净增至 98 个，仿真失败率从 28.2% 降至 16.2%。这表明仿真奖励引导模型学会了更规范的代码生成模式，有效减少了语法与结构性错误。

仿真成功样本的 F1 变化。仅考虑通过仿真的样本，SFT 模型的平均 F1 为 0.902，而 GRPO 模型为 0.826。这一下降的原因在于：GRPO 新增通过仿真的 14 个样本大多为 SFT 阶段完全失败（F1=0）的困难样本，虽然现在能够通过编译，但功能正确性尚未达到高水平，拉低了仿真成功子集的平均分。整体 F1 仍然从 0.648 提升至 0.692，表明新增样本的贡献超过了个体 F1 的下降。

优化方向的验证。230 步的 GRPO 训练将整体 F1 从 0.648 提升至 0.692，主要贡献来自将更多样本从“完全失败”转化为“部分正确”。这一结果验证了基于仿真奖励的强化学习作为 SFT 后续优化手段的可行性，同时也表明在有限的训练步数下，模型的优化主要体现在代码结构规范性的改善上，功能逻辑层面的深层优化仍需更充分的训练。

与通用大模型的对比。结合表 4.4 的完整结果，从 Qwen3-8B 零样本的 0.263 到 SFT 后的 0.648，再到 GRPO 后的 0.692，本文的训练流水线使一个 8B 参数模型逐步逼近了 Claude Opus 4.6 闭源模型（F1=0.744）。尤其值得注意的是，GRPO 将通过率从 84/117 提升至 98/117，与 Claude 完全持平，表明基于仿真奖励的强化学习

能够有效弥补小模型在代码结构规范性上的不足。仅考虑通过仿真的样本，Claude Opus 的平均 F1 (0.889) 高于 GRPO 模型 (0.826)，反映了通用大模型凭借更强的代码生成能力在功能逻辑准确性上仍有优势，也意味着继续扩大 RL 训练规模或引入更精细的奖励信号仍有提升空间。

从工程应用角度，8B 参数模型可在单张消费级 GPU 上完成推理，适合集成于对响应速度与调用成本敏感的场景；而本文的训练方法论表明，通过高质量数据合成与针对性强化学习，小规模开源模型可以达到接近前沿闭源模型的性能水平，同时保持完全的本地部署自主权。

4.5.3 典型改善案例

为直观展示 GRPO 训练的具体改善效果，本节列举三个 SFT 模型编译失败但 GRPO 模型成功生成可仿真代码的典型样本。

案例一：超声发射脉冲控制器 (F1: 0 → 0.951)。该模块包含 6 个信号，输入包括时钟、接收门控、发射延迟参数与脉宽参数，输出为差分发射脉冲对 TXP/TXN。SFT 模型生成了包含约 80 条语句的单块代码，声明了 17 个寄存器与 12 个 `parameter` 常量，试图实现一个包含多状态的发射控制有限状态机，但其中引用了未声明的变量（如 `PulseCtrNext`），导致编译失败。GRPO 模型生成了 23 行简洁代码，虽然逻辑实现为简化的门控赋值而非完整状态机，但端口声明规范、语法正确，仿真波形与标准答案的 F1 达到 0.951。

案例二：存储器仲裁状态机 (F1: 0 → 0.936)。该模块为一个包含 13 个信号的存储器访问仲裁器，需要在 CRT 显示请求、CPU 读请求与 CPU 写请求之间进行优先级调度。SFT 模型生成的代码结构完整，包含正确的状态定义、状态转移逻辑与组合输出，但将三个输出端口 (`crt_gnt`、`cpu_wr_gnt`、`cpu_rd_gnt`) 声明为 `output reg` 后又使用 `assign` 连续赋值驱动，违反了 Verilog 语法规则 (`reg` 类型不能被 `assign` 驱动)，导致编译失败。GRPO 模型将输出逻辑改为在 `always` 块中以时序逻辑赋值，避免了类型冲突，成功通过编译与仿真。

案例三：处理器控制单元 (F1: 0 → 0.880)。该模块是一个包含 11 个信号的指令译码控制单元，输入为 5 位操作码 `opcode`，输出包括 PC 多路选择、数据通路控制、ALU 操作码、寄存器写使能等多个控制信号。SFT 模型生成了 253 行代码，包含完整的 `case` 译码逻辑，但在定义操作码常量时重复声明了同一标识符（如 `SHROP`、`JMPOP` 分别作为 `parameter` 和 `localparam` 各声明一次），导致编译失败。GRPO 模型生成了 244 行代码，正确避免了重复声明，成功通过编译与仿真。

上述案例呈现出一致的改善模式：SFT 模型在复杂模块上倾向于生成过度复

杂的代码（更多的寄存器声明、更多的状态定义、更多的中间变量），而这些复杂结构中往往包含类型冲突或未声明变量等结构性错误。GRPO 训练通过仿真奖励引导模型学会了“可编译优先”的生成策略：倾向于生成结构更简洁、类型声明更规范的代码，即使逻辑上有所简化，也优先确保代码能够通过编译并产出仿真结果。这一策略偏移直接反映了仿真奖励函数的设计：编译失败的代码获得零奖励，而任何能够通过仿真的代码即使 F1 不高也能获得正奖励，使模型在探索过程中逐步学会避免结构性错误。

4.6 本章小结

本章通过系统的实验验证了本文所提框架的有效性。数据集统计分析表明，合成数据在信号复杂度、代码长度与模块功能类型上具备良好的多样性。评价指标对照实验证实了本文指标相比传统文本相似度指标在结构语义评估上的优越性。第一阶段微调模型以 4B 参数达到 0.969 的平均 F1，远超参数量高出数十倍的通用多模态模型；第二阶段微调模型以 8B 参数超越全部开源基线，经 GRPO 优化后在仿真通过率上与 Claude 闭源模型持平，整体 F1 差距收窄至 0.052。错误分析揭示了模型的主要瓶颈，即从行为观测推断逻辑规则的能力不足，以及强化学习改善代码结构规范性的具体机制。

第 5 章 总结与展望

5.1 工作总结

本文围绕数字电路时序图的自动化解析问题，提出了一种基于多模态大语言模型的两阶段解析框架，实现了从时序图图像到 WaveJSON 结构化表示、再到可综合 Verilog 代码的完整处理流水线。

在数据构建方面，本文针对该领域缺乏标注数据的核心瓶颈，设计了一套以 Verilog 仿真为基础的自动化数据合成链路。通过引入大语言模型替代随机激励生成具有功能覆盖性的测试平台，使合成的时序图能够充分展示模块的关键工作状态；同时利用随机管线的产出作为低成本探针对候选模块进行质量预选，并采用基于并查集的防泄漏划分策略确保评估的可靠性，最终从 108,971 个原始 Verilog 源文件中筛选出 5,847 个高质量训练样本。

在评价体系方面，本文提出了面向 WaveJSON 结构特性的信号级 F1 评价指标，通过信号名匹配、保持符号展开与时钟等价性处理，解决了传统文本相似度指标对信号顺序敏感且无法区分语义错误严重程度的问题。该指标同时作为强化学习的奖励函数发挥了双重作用。

在模型训练方面，本文将视觉理解与代码推理解耦为两个独立阶段：第一阶段使用 Qwen3-VL-4B-Instruct 通过 LoRA 微调实现时序图到 WaveJSON 的转换，在 117 个测试样本上达到 99.1% 的解析成功率与 0.969 的平均 F1；第二阶段使用 Qwen3-8B 通过 LoRA 微调实现 WaveJSON 到 Verilog 代码的转换，以 8B 参数量超越了参数量高出数十倍的开源通用模型。在此基础上，本文引入 GRPO 强化学习以仿真对拍的 F1 分数作为连续值奖励信号，将仿真通过率从 71.8% 提升至 83.8%，整体 F1 从 0.648 提升至 0.692，与 Claude Opus 4.6 闭源模型（0.744）差距收窄至 0.052，充分验证了面向特定任务的数据合成与强化学习训练流水线的有效性。

5.2 局限性与未来展望

本文方法仍存在若干局限性，同时也指明了未来的研究方向。

其一，训练与评测数据均为合成生成的 WaveDrom 风格时序图，模型对真实工程场景中的时序图（如芯片数据手册中的协议规范图、EDA 工具导出的仿真波形截图或手绘草图）的泛化能力尚未验证。未来可从数据手册、IP 规格文档等来源采集真实时序图数据，结合多种渲染风格与分辨率进行跨域训练，弥合合成数

据与实际应用之间的分布差异。

其二，两阶段流水线存在误差传播的固有风险——第一阶段的信号名错误或波形长度偏差会直接劣化第二阶段的代码质量。未来可探索端到端的单阶段架构，或引入跨阶段反馈机制（如利用第二阶段的编译错误信息引导第一阶段修正输出）以缓解误差累积。

其三，基于强化学习的优化仍处于初步探索阶段，且训练尚不充分。当前的探索主要验证了仿真奖励作为优化信号的可行性，但模型的改善集中在代码结构规范性上，功能逻辑层面的深层能力仍有较大提升空间。未来可在奖励函数设计（如结合形式化验证工具提供更强的正确性保证）、训练策略与规模等方面进行更深入的探索。

其四，当前数据合成依赖单一的 Verilog 源代码库，模块类型以中小规模 RTL 设计为主。未来可将数据来源扩展到工业级 IP 设计，覆盖更复杂的参数化结构与总线协议交互，并将本文方法应用于设计文档与 RTL 实现的一致性检查等实际工程场景，最终集成到 EDA 工具链中构建智能设计辅助系统。

参考文献

- [1] AMBA AXI and ACE protocol specification[M]. Arm, 2011.
- [2] AMBA AHB protocol specification[M]. Arm, 2021.
- [3] AMBA APB protocol specification[M]. Arm, 2021.
- [4] VASWANI A, et al. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017, 30.
- [5] OpenAI. Introducing GPT-5.5[EB/OL]. 2026. <https://openai.com/index/introducing-gpt-5-5/>.
- [6] Meta. The Llama 4 herd[EB/OL]. 2025. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [7] YANG A, et al. Qwen3 technical report[A]. 2025.
- [8] DeepSeek-AI. DeepSeek-V4 technical report: Advancing open-source language models[J]. Technical report, 2025.
- [9] LIU M, PINCKNEY N, KHAILANY B, et al. VerilogEval: Evaluating large language models for Verilog code generation[C]//Proceedings of ICCAD. IEEE, 2023: 1-8.
- [10] THAKUR S, et al. Benchmarking large language models for automated Verilog RTL code generation[C]//Proceedings of DATE. IEEE, 2023: 1-6.
- [11] THAKUR S, et al. VeriGen: A large language model for Verilog code generation[J]. ACM Transactions on Design Automation of Electronic Systems, 2024, 29(3): 1-31.
- [12] LIU M, et al. ChipNeMo: Domain-adapted LLMs for chip design[A]. 2023.
- [13] FU Y, et al. GPT4AIGChip: Towards next-generation AI accelerator design automation via large language models[C]//Proceedings of ICCAD. IEEE, 2023: 1-9.
- [14] BLOCKLOVE J, et al. Chip-chat: Challenges and opportunities in conversational hardware design[C]//Proceedings of MLCAD. IEEE, 2023: 1-6.
- [15] NVIDIA. LLM4HWDDesign[EB/OL]. 2024. <https://nvlabs.github.io/LLM4HWDDesign/>.
- [16] OpenAI. GPT-5 system card[A]. 2025.
- [17] Gemini Team, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities[A]. 2025.
- [18] LIU H, LI C, WU Q, et al. Visual instruction tuning[J]. Advances in Neural Information Processing Systems, 2023, 36.
- [19] WANG W, et al. InternVL3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency[A]. 2025.
- [20] BAI S, et al. Qwen3-VL technical report[A]. 2025.
- [21] HE J, NIČKOVIĆ D, BARTOCCI E, et al. TD-Magic: From pictures of timing diagrams to formal specifications[C]//Proceedings of DAC. IEEE, 2023: 1-6.
- [22] HE J, KENBEEK V T W, YANG Z, et al. TD-Interpreter: Enhancing the understanding of timing diagrams with visual-language learning[A]. 2025.

- [23] CHANG K, et al. Natural language is not enough: Benchmarking multi-modal generative AI for Verilog generation[C]//Proceedings of ICCAD. ACM/IEEE, 2024.
- [24] SHAO Z, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models[A]. 2024.
- [25] RADFORD A, et al. Language models are unsupervised multitask learners[J]. OpenAI Blog, 2019.
- [26] BROWN T, et al. Language models are few-shot learners[J]. Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.
- [27] ACHIAM J, et al. GPT-4 technical report[A]. 2023.
- [28] TOUVRON H, et al. Llama 2: Open foundation and fine-tuned chat models[A]. 2023.
- [29] GRATTAFIORI A, et al. The Llama 3 herd of models[A]. 2024.
- [30] GLM Team, et al. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models[A]. 2025.
- [31] OpenAI. gpt-oss-120b & gpt-oss-20b model card[A]. 2025.
- [32] DOSOVITSKIY A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[C]//ICLR. 2021.
- [33] RADFORD A, KIM J W, HALLACY C, et al. Learning transferable visual models from natural language supervision[C]//ICML. 2021: 8748-8763.
- [34] CHEN Z, et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks[C]//Proceedings of the IEEE/CVF CVPR. 2024: 24185-24198.
- [35] OpenAI. GPT-4o system card[A]. 2024.
- [36] Gemini Team, et al. Gemini: A family of highly capable multimodal models[A]. 2023.
- [37] BAI S, et al. Qwen2.5-VL technical report[A]. 2025.
- [38] WaveDrom. WaveDrom: Digital timing diagram rendering engine[EB/OL]. 2024. <https://wavedrom.com>.
- [39] WILLIAMS S. Icarus Verilog compilation system[EB/OL]. 2024. <https://github.com/steveicarus/iverilog>.
- [40] Toroid-io. vcd2wavedrom: Transform VCD to WaveDrom[EB/OL]. 2024. <https://github.com/Toroid-io/vcd2wavedrom>.
- [41] WaveDrom. wavedrom-cli[EB/OL]. 2024. <https://github.com/wavedrom/cli>.
- [42] PAPINENI K, ROUKOS S, WARD T, et al. BLEU: A method for automatic evaluation of machine translation[J]. Proceedings of the 40th Annual Meeting of the ACL, 2002: 311-318.
- [43] LIN C Y. ROUGE: A package for automatic evaluation of summaries[C]//Text Summarization Branches Out. 2004: 74-81.
- [44] LEVENSHTAIN V I. Binary codes capable of correcting deletions, insertions, and reversals[J]. Soviet Physics Doklady, 1966, 10(8): 707-710.
- [45] HU E J, et al. LoRA: Low-rank adaptation of large language models[C]//ICLR. 2022.
- [46] SCHULMAN J, et al. Proximal policy optimization algorithms[A]. 2017.

- [47] FISLER K. Timing diagrams: Formalization and algorithmic verification[J]. Journal of Logic, Language and Information, 1999, 8(3): 323-361.
- [48] MALER O, NICKOVIC D. Monitoring temporal properties of continuous signals[C]//LNCS: Vol. 3253 Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems. Springer, 2004: 152-166.
- [49] BARTOCCI E, MATEIS C, NESTERINI E, et al. Survey on mining signal temporal logic specifications[J]. Information and Computation, 2022, 289: 104957.
- [50] EDAUtils. Verilog testbench generator[EB/OL]. 2024. <https://www.edautils.com/VlogTBGen.html>.
- [51] CHIANG W L, et al. Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality[EB/OL]. 2023. <https://lmsys.org/blog/2023-03-30-vicuna/>.
- [52] ZHAO Y, GU A, VARMA R, et al. PyTorch FSDP: Experiences on scaling fully sharded data parallel[J]. Proceedings of the VLDB Endowment, 2023, 16(12): 3848-3860.
- [53] ZHENG Y, ZHANG R, ZHANG J, et al. LlamaFactory: Unified efficient fine-tuning of 100+ language models[C]//Proceedings of the 62nd Annual Meeting of the ACL: System Demonstrations. 2024: 400-410.
- [54] SHENG G, ZHANG C, YE Z, et al. HybridFlow: A flexible and efficient RLHF framework[C]//Proceedings of the Twentieth European Conference on Computer Systems (EuroSys). ACM, 2025: 755-771.
- [55] KWON W, LI Z, ZHUANG S, et al. Efficient memory management for large language model serving with PagedAttention[C]//Proceedings of the 29th Symposium on Operating Systems Principles. 2023: 611-626.
- [56] ANTHROPIC. The claude model family[EB/OL]. 2025. <https://www.anthropic.com/research/claude-model-family>.
- [57] Qwen Team. Qwen3-Coder technical report[J]. Technical report, 2025.

附录 A 外文资料的书面翻译

TD-Interpreter——通过视觉-语言学习增强时序图理解

摘要：本文介绍 TD-Interpreter，一个旨在帮助工程师在设计与验证过程中理解来自第三方的复杂时序图（Timing Diagrams, TDs）的专用机器学习工具。TD-Interpreter 是一个视觉问答环境，允许工程师输入一组时序图并就这些时序图提出设计和验证方面的问题。我们通过多模态学习对 LLaVA——一个轻量级的 70 亿参数多模态大语言模型（MLLM）——进行微调来实现 TD-Interpreter。为了解决训练数据有限的问题，我们开发了一套合成数据生成流程，实现视觉信息与文本解释的对齐。实验评估证明了 TD-Interpreter 的有效性，它在所评估的基准测试中大幅优于未经微调的 GPT-4o。

关键词：数字设计，时序图，多模态大语言模型，视觉-语言学习

A.1 引言

数字设计是一个复杂的过程，涉及使用硬件描述语言将需求转化为数字电路。虽然高层次综合方法和先进的 IP 核降低了设计过程的时间与复杂度，但工程师对各种硬件组件功能的熟悉程度仍然至关重要。时序图作为描述不同信号之间功能与交互的视觉化需求规范，在设计与验证过程中发挥着核心作用。因此，对时序图的深入理解是至关重要的。

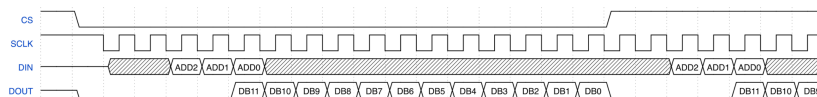
不同的时序图在抽象层次上有所不同。文档和数据手册中经常包含描述基本时序约束或特定通信协议操作逻辑的通用高层次时序图。从现有代码生成的时序图往往提供了来自资深工程师的详细见解。工程师在日常工作中绘制的时序图介于这两个层次之间。为了帮助工程师更好地理解时序图，本文提出了 TD-Interpreter，一个能够在多种场景下解释时序图的 AI 工具，如图 A.1 所示。

第一个动机示例涉及在初始设计阶段理解一组抽象时序图。图 A.1(A) 展示了 TD-Interpreter 如何帮助一位正在设计 FPGA 与模数转换器（ADC）接口的工程师。如图 A.1(A) 所示，工程师从数据手册 [1] 中上传时序图图片，并询问其规范，例如重要的时序事件。TD-Interpreter 随后可以自动分析和解释该时序图。

第二个动机示例涉及在后期设计阶段理解具体时序图。图 A.1(B) 展示了这样一个使用场景：工程师需要为专用硬件定制第三方异步先进先出（FIFO）模块。工程师可以通过生成该模块的时序图图片，并将其连同相关问题上传到 TD-Interpreter

A

What are the important timing events for the ADC interface?



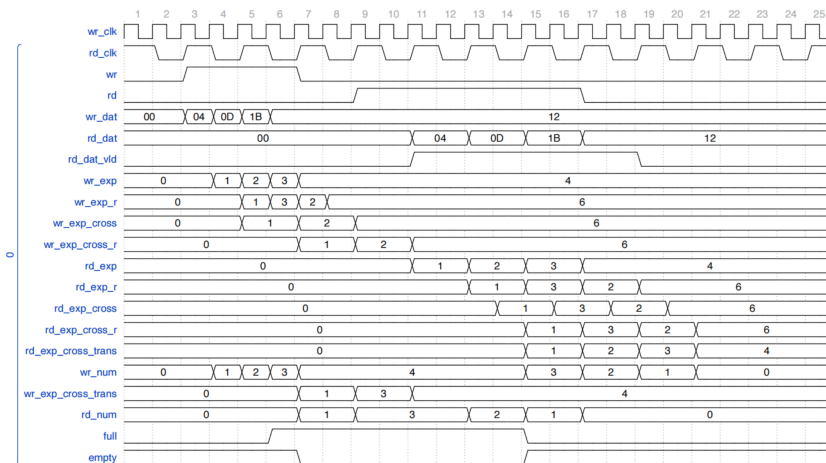
- (1) After signal "CS" goes down, the FPGA should send the first falling edge of "SCLK" to the ADC;
- (2) The FPGA should send address bit "DIN" to the ADC at 3rd, 4th, 5th falling edges of "SCLK", which enables the ADC to stably read these bits at the 3rd, 4th, 5th rising edges of "SCLK";
- (3) The ADC sends effective "DOUT" bit to the FPGA at 5th-16th falling edges of ADC, which enables the FPGA to stably read each "DOUT" bit at 5th-16th rising edges of "SCLK".

B

Start a new conversation, and the history is cleared.



- (1) What data transaction procedure does this TD show?
- (2) How do signals correlate each other in different clock domains?
- (3) How to count the number of data inside the FIFO?
- (4) How to signify the FIFO is full?



TD-Interpreter is typing

图 A.1 与 TD-Interpreter 的对话

来熟悉现有模块。

TD-Interpreter 的开发得益于多模态大语言模型 (MLLMs) 的进展, 这些模型已经实现了文本生成图像、图像描述和视觉问答等多样化应用。然而, 直接使用 GPT-4o 等商业 MLLM 进行时序图解释仍然存在问题, 原因包括知识产权隐私顾虑、微调受限以及训练数据不足。为了解决这些问题, 我们探索了面向资源有限且关注知识产权保护的企业量身定制的低成本数据构建、训练与本地部署方案。具体而言:

1. 我们开展了一项以人为中心的实证研究, 对 31 位嵌入式系统和数字设计领域的专家进行了访谈。我们收集了关于他们使用时序图的专业经验、工作中遇到的挑战与困难、对实际时序图的反馈以及对 TD-Interpreter 功能期望的统计信息。
2. 我们通过基于实证研究开发自动化的合成数据生成器, 解决了数据集匮乏的问题, 将时序图解释转化为多模态学习中的视觉问答任务。我们生成了两类数据: 第一类侧重于为给定时序图生成描述, 增强 MLLM 描述时序图内部细节信息的能力; 第二类针对分析任务, 使 MLLM 能够对多种时序关系进行推理。
3. 我们通过采用描述增强的视觉推理方法成功微调开源 MLLM LLaVA[2], 解决了知识产权保护和微调受限的问题。通过在训练中结合描述与推理任务, LLaVA 在时序图细节推理方面展现出令人期待的能力。该模型仅有 70 亿参数, 非常适合本地部署。

总而言之, TD-Interpreter 将数字设计领域的专业知识与多模态学习的进展相结合, 促进了时序图的自动分析, 尤其将行业中的隐性知识数字化, 从而加速整体开发过程。

本文组织如下: 第 A.2 节介绍相关工作, 第 A.3 节展示实证研究的细节, 第 A.4 节说明训练数据的生成方式, 第 A.5 节探讨多模态 LLM 的微调方法, 第 A.6 节介绍和分析实验结果。

A.2 相关工作

自大语言模型 (LLM) 问世以来, 已有多项研究探索了其在辅助硬件设计方面的应用。例如, 文献 [3-7] 研究了将文本设计需求转换为 Verilog 代码。这些工作要么在商业 LLM 中使用提示工程, 要么微调开源 LLM, 如 Llama[8] 和 CodeGen[9]。也有一些工作将 LLM 的应用扩展到其他领域, 例如加速器设计 [10]。然而, 上述工作均试图将问题转化为自然语言处理 (NLP) 的代码生成任务, 因此这些工作中

使用的 LLM 仅处理文本数据。本文提出的视觉问答（VQA）任务则通过视觉-语言学习探索了 MLLM 的潜力，使 TD-Interpreter 能够同时处理视觉和文本两种模态的数据。最近，文献 [11] 也考虑了将 MLLM 应用于硬件设计，但该工作并未关注时序图，且目标仍为代码生成。

A.2.1 时序图

在近期工作中，文献 [12] 提出了 TD-Magic，该方法可以从时序图图像中提取严格偏序（SPO）关系作为形式化规范。然而，由于只有极少数时序图包含明确的 SPO 关系，TD-Magic 的应用范围主要局限于为晶圆代工厂的电路组件创建库文件或为 PrimeTime[13] 等工具创建时序验证脚本。此外，TD-Magic 的多模块、基于规则的设计也限制了其处理大规模时序图图片的能力。相比之下，TD-Interpreter 借助 MLLM 在设计和验证中服务于更广泛的工程师群体，通过视觉问答环境使他们能够解释更复杂的时序图。然而，TD-Magic 和 TD-Interpreter 的共同之处在于，两者都模拟了工程师阅读数字电路时序图图像的过程，这使它们区别于另一类直接分析可能包含混合信号波形数据点的研究（用于测试或监控目的）[14]。这些工作采用模型到模型的方法，将数据点转换为自动机 [15] 和时序逻辑 [16,17] 等形式语言。

A.2.2 多模态大语言模型

多模态大语言模型（MLLMs）能够处理来自多种数据模态（如文本、图像或音频）的信息，并以其他模态提供输出 [18]。LLaVA[2] 作为最流行的开源 MLLM 之一，采用了基于视觉 Transformer（ViT）[19] 的视觉编码器、基于 LLaMA[8] 的语言模型，以及一个简单的线性层作为适配器来减少训练参数。近年来，MLLM 通过使用更大更强的文本编码器或适配器 [18] 以及改进的对齐方法 [20] 得到了增强。MLLM 已被证明是执行图像描述、视觉问答和视觉推理等多模态任务的强大工具。然而，将其应用于技术图表等领域特定图像仍具有挑战性。在本文中，我们解决了这一问题，并证明了像 LLaVA 这样的轻量级 MLLM 可以作为复杂任务的稳健解释器。

A.3 以人为中心的实证分析

为确保系统设计能够反映工程师在实际中遇到的挑战，我们对 31 位专业人员进行了以人为中心的实证分析，了解他们使用 AI 工具解释时序图的情况。以下是我们的结果。

A.3.1 参与者背景

大多数参与者来自大学、研究机构或硬件公司。在所有参与者中，3 人（9.7%）拥有学士学位，11 人（35.5%）拥有硕士学位，17 人（54.8%）拥有博士学位或正在攻读博士学位，其中包括三位计算机工程教授。

这些参与者涵盖了硬件设计方面广泛的专业技能，包括数字电路设计、FPGA 开发、ASIC 开发、硬件验证和嵌入式系统。

A.3.2 时序图使用经验

我们询问了参与者在什么场景下通常使用时序图，有趣的是，我们发现了多种使用案例。例如，超过一半的参与者（18 人，58.1%）提到使用时序图来理解通信协议（如 SPI、I2C、UART）。10 人（32.2%）强调他们依赖时序图来调试新硬件设计，或理解甚至逆向工程来自同事或第三方的现有设计。4 人（12.9%）从事硬件验证工作，特别使用时序图来编写和验证 Verilog/VHDL 测试平台。8 人（25.8%）强调了使用时序图与他人沟通的重要性，例如展示设计、小组讨论和团队协作。三分之二的参与者告诉我们他们在职业初期使用过时序图，例如课程作业或工业实习项目。这些发现与我们在第 A.2 节中解释时序图的动机一致。

我们进一步询问了参与者通常在何处接触时序图，他们的回答可以分为两组，类似于第 A.2 节中的动机示例。从他们的回答来看，一半的资源是来自数据手册、内部设计文档或教科书的抽象时序图；另一半是来自 Verilog/VHDL 代码或 MATLAB/Simulink 仿真的具体波形。

由于决定支持哪种格式的时序图很重要，我们专门设计了一个问题来询问参与者使用什么工具来创建或分析时序图。有趣的是，我们仍然收到了两组均匀分布的答案。第一组人提到“他们主要手动绘制时序图”。第二组资源是来自 GTKWave 和 ModelSim 等仿真工具的波形。尽管参与者提到了多种时序图格式，但在工业界，人们倾向于使用标准格式以便于沟通，其中 WaveDrom 应用最为广泛。如第 A.4 节将要介绍的，WaveDrom 支持以脚本方式绘制任意复杂度的标准时序图，并提供将仿真波形直接转换为其标准格式的接口。鉴于这些优势，我们选择解释以 WaveDrom 格式表示的时序图。

A.3.3 挑战与困难

在这组问题中，参与者被要求使用 1（非常容易）到 5（非常困难）的量表对理解时序图时的常见挑战进行评分。以下是各项挑战及其平均难度评分：

- C1：从数据手册中提取有意义的信息。（2.97/5.0）

- C2: 理解通信协议的整体事务流程。(3.16/5.0)
- C3: 理解信号之间的输入/输出关系。(2.84/5.0)
- C4: 从信号行为中提取关键事件, 例如某个上升/下降沿的含义。(2.77/5.0)
- C5: 理解多个时钟周期内多个信号之间的关系。(3.68/5.0)
- C6: 识别多时钟系统中的时序关系/约束。(3.77/5.0)
- C7: 分析跨时钟域行为。(3.84/5.0)
- C8: 测量时序参数 (如延迟和延时) 并识别建立时间和保持时间违规。(3.26/5.0)
- C9: 检测亚稳态相关问题。(4.0/5.0)
- C10: 将时序图与实际硬件行为匹配。(3.23/5.0)

平均分高于 3 的挑战被认为对工程师来说相对困难, 这与数字设计领域的普遍经验一致。特别是, 理解跨多个信号和时钟周期的时序关系, 以及识别跨时钟域 (CDC) 场景中与亚稳态相关的问题, 被广泛认为是复杂且常常难以处理的任务。

除了上述困难外, 一些参与者还补充说, 从波形轨迹中提取有限状态机具有挑战性, 这在分析通信协议时是一项常见任务。这些发现将指导我们在第 A.4 节中详述的训练数据生成过程。

A.3.4 AI 的使用

在本小节中, 我们调查了参与者对开发 TD-Interpreter 作为辅助其工作中时序图解释的潜在工具的态度。绝大多数参与者 (30/31) 表示如果这样的工具存在, 他们愿意使用它, 其中 24 位参与者对其应用表达了明确的兴趣。超过 90% 的参与者期望 TD-Interpreter 在回答问题时展示推理过程。

此外, 当被要求设想 TD-Interpreter 的潜在使用案例时, 参与者建议了以下有用的功能:

- 时序图的自动解释 (21 人, 67.7%)
- 解释图中的逐步转换 (22 人, 71%)
- 比较多个时序图 (17 人, 54.8%)
- 解释模糊标注 (16 人, 51.6%)
- 检查时序违规 (18 人, 58.1%)
- 生成设计和验证建议 (8 人, 25.8%)
- 基于时序图提供 Verilog 代码建议 (9 人, 29%)

最后, 我们询问了参与者对 TD-Interpreter 首个演示版本的期望。最受期待的三个应用领域为: (1) 帮助解决 HDLBits 网站 [21] 上的习题, 主要因为该平台是练习硬件设计技能的知名平台; (2) 解答与常见外设协议 (如 UART、I2C、SPI)

时序图相关的问题，主要因为这些协议被初级硬件设计师广泛使用且在嵌入式系统开发中很常见；（3）解答与总线协议（如 AHB 或 AXI）时序图相关的问题，因为这些协议通常由更有经验的工程师接触，涉及丰富的时序关系。

此外，两位资深参与者指出，如果 TD-Interpreter 能够从波形中综合状态机并识别与跨时钟域（CDC）相关的潜在问题，将更加有用。

A.4 数据集构建

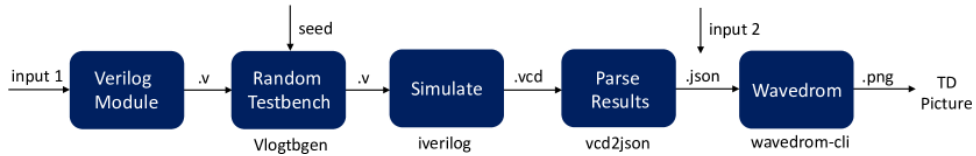


图 A.2 生成时序图图片的工作流

微调 MLLM 以解释时序图需要一个带注释的时序图数据集，但目前不存在此类数据集。因此，我们开发了一种生成大规模时序图数据集的策略，每个时序图配有用于视觉指令调优的问答（QA）对。

图 A.2展示了我们生成时序图图片的工作流。第一条路径获取具体时序图图片，从图 A.2中的“输入 1”开始，解析 Verilog 模块以提取基本信息，如其输入和输出端口。这些信息传递给 EDAUtils 的测试平台生成器 Vlogtbgen[22]，为模块端口生成提供随机输入值的测试平台。生成的测试平台使用 iverilog[23] 编译和仿真，产生包含仿真结果的.vcd 文件。我们使用 vcd2json[24] 解析.vcd 文件，获得包含时序图内所有必要图形信息的.json 字典。最后，wavedrom-cli[25] 工具使用此.json 文件创建 WaveDrom 格式的时序图图片。该图形格式在数字设计中被广泛使用。对于生成抽象时序图图片，我们直接从“输入 2”开始，手动提供来自数据手册中时序图的.json 内容。

我们的数据生成方法包含两个阶段。在第一阶段，我们专注于生成基于描述的数据，这些数据以为给定时序图图片提供描述为中心，旨在帮助 MLLM 捕获时序图内部的细节。第二阶段专注于生成基于推理的数据，这些数据基于实际使用案例，旨在增强 MLLM 对时序图内部多种时序关系的推理能力。

A.4.1 基于描述的数据生成

A.4.1.1 方法

图 A.2中展示的策略允许我们从 Verilog 模块生成时序图，无需手动输入。将其相关流程应用于公开可用的数据集（如文献 [5]）可支持快速生成时序图。那 QA

对呢？

来自.vcd 文件的仿真结果可以用于自动创建相对简单的 QA 对，适合训练 MLLM 理解时序图的基础知识。例如，.vcd 文件中包含每个信号 s 在时钟周期 n 处的值。解析这些数据可以自动创建如“信号 s 在时钟周期 n 的值是什么？”的 QA 对。可以生成的其他 QA 对还包括，例如“信号 s 的值序列是什么？”、“信号 s 有多少次转换？”以及“信号 s 有多少个上升沿？”。这些问题侧重于对信号时序特征的基本理解，为更复杂的 QA 对提供了坚实的基础。

更高级的 QA 对通常难以通过编程生成。然而，对于某些 Verilog 模块，我们成功生成了更抽象的 QA 对，例如“描述一下你在这个时序图中看到的内容”。这是可能的，因为其中一些模块具有高度描述性的名称。通过利用这些描述性名称和模块端口的可用信息，我们使用了 DeepSeek-Coder-V2[26]——一个专为代码相关任务设计的、支持 Verilog 代码的 LLM。

借助生成知识提示技术 [27]，DeepSeek-Coder-V2 围绕给定的 Verilog 模块构建上下文。这是通过思维链提示 [28] 实现的，其中 LLM 由一组子任务引导，以小步骤分析所提供的模块。以下提示用于构建这一思维链提示过程：“你的任务是根据有限的数字组件提供简要描述。对于每个组件，提供该组件的描述。为此请执行以下步骤：1. 假设它是一个 Verilog 模块。2. 按‘_’字符分割模块名称。3. 列出并分析模块名称的各个部分。4. 列出并计数输入和输出端口。5. 分析输入和输出端口的名称。6. 确定每个端口的可能功能。7. 根据名称的各个部分和 I/O 端口确定模块的功能。”

按照此提示，LLM 能够提供清晰而详细的回答，为所提供的 Verilog 模块生成描述、摘要和使用案例列表。我们还能够生成以下问题：“提供时序图的描述/标题/摘要/使用案例”。

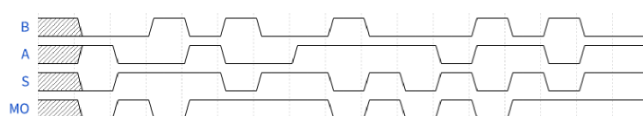


图 A.3 2 选 1 负多路复用器的时序图

我们训练策略的一个示例是图 A.3 中所示的时序图所获得的结果，该图表示一个名为“maxv_nmux21”的 Verilog 模块。通过上述提示过程，DeepSeek-Coder-V2 成功识别该模块为 2 选 1 负多路复用器，提供了详细描述，解释了其目的和输入/输出端口，为时序图配上了合适的标题，给出了简要摘要和使用案例列表。为了突出此类训练数据的必要性，我们将同一时序图提供给 GPT-4o 并要求其解释。它只回复了基本的信号枚举和高/低切换的通用解释，将 A 和 B 输入信号错误识别为时钟信号，并将该图误解为可能表示某种时钟信号关系或状态机。

A.4.1.2 统计

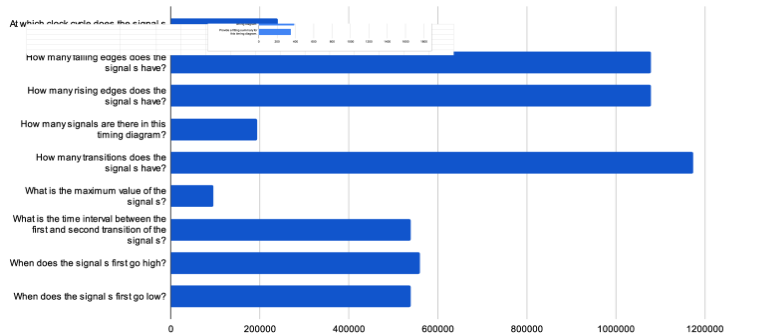


图 A.4 问题类型分布

通过上述基于描述的数据生成方法，共生成了 221,983 个时序图。对于这些时序图，我们生成了 9,915,694 个 QA 对，其中 4,306,008 个是“信号 s 在时钟周期 n 的值是什么？”的形式，需要 MLLM 检测特定时间点的具体信号值。其他问题的分布如图 A.4 所示。此外，对于 28,271 个时序图，相应 Verilog 模块的名称具有足够的描述性，可以使用 DeepSeek-Coder-V2 生成额外的描述。

A.4.2 基于推理的数据生成

与可以部分通过 EDA 工具自动化的基于描述的数据不同，基于推理的数据的生成高度依赖对第 A.3 节实证分析结果的人工审查，包括使用案例的选择和工程师在使用时序图时面临的典型挑战的考量。

我们确定了一组常用的系统设计组件，包括存储器、通信协议和定时器。我们还考虑了 HDLBits 网站 [21] 上的设计题目，这些题目是广泛认可的评估 LLM 代码生成能力的基准。对于每个组件，我们手动走过每个设计步骤，特别关注设计者容易犯微妙错误的关键时序点。我们还有意设计了非常具体甚至非常规的问题，这些问题需要高度依赖经验和对具体上下文的理解才能解答。

为了演示我们如何从具体时序图图片生成 QA 对，我们首先使用串行接收器协议 [29] 的设计。这是一个说明性示例，因为它具有一定的复杂性。相同的方法可以扩展到其他使用案例。随后我们简要讨论了为抽象时序图生成数据的方法，该方法更为简单。我们还提供了生成数据的统计分析。

A.4.2.1 生成 QA 对

串行接收器示例描述了一个通信协议，接收字节流并带有额外的第 9 位和第 10 位分别用于奇偶校验和停止检查。正确接收一个字节的的要求为：(R1) 前 9 位应包含奇数个 1；(R2) 第 10 位为 1。此外，工程师通常会看到如图 A.5 所示的期望时序图。

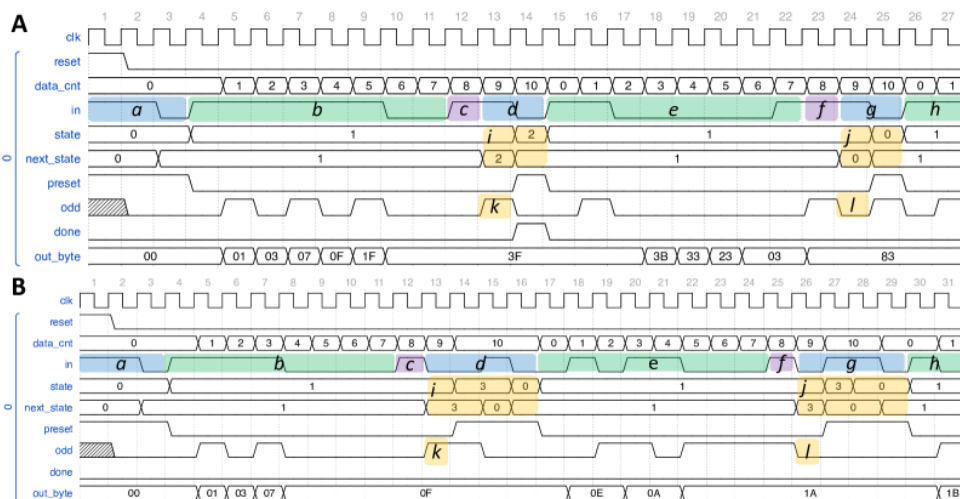


图 A.5 串行接收器协议的期望时序图

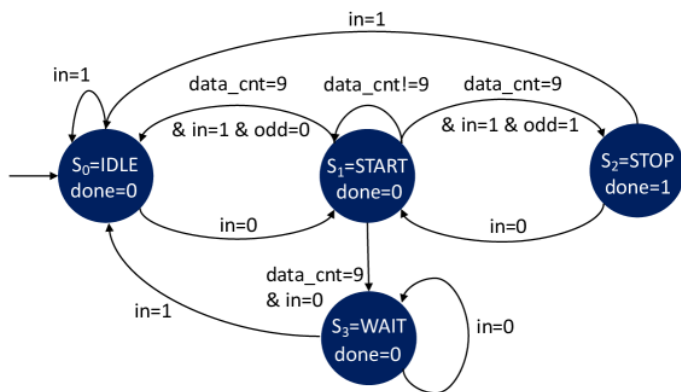


图 A.6 串行接收器的 Moore 有限状态机

伴随文本需求，典型的设计工作流需要工程师分析和推理期望时序图。我们的 QA 对构建方法也遵循此过程。

有限状态机设计 图 A.6展示了表示上述协议事务逻辑的有限状态机（FSM）。图 A.5(A) 展示了成功场景，状态在 S_0 、 S_1 、 S_2 之间转换；图 A.5(B) 展示了失败场景，状态在 S_0 、 S_1 、 S_3 之间转换。根据文本需求和期望时序图构建部分/完整的 FSM 是数字设计中的经典问题，我们对此给予了特别考虑。

数据通路设计 如图 A.6所示， $data_cnt = 9$ 是状态转换的关键条件，因此像 $data_cnt$ 这样的计数器信号设计非常重要。给定期望时序图，我们可能会问：在哪些场景下 $data_cnt$ 会变为 0、保持不变或递增？此外，如图 A.5所示，信号 out_byte 与信号 in 中的序列密切相关，我们还可以提问： out_byte 如何从 in 中顺序存储位？

关键时序分析 由于工程师经常对波形的特定行为感到困惑，这类问题专门为此场景设计。它们还可以用于增强 MLLM 对组合逻辑和时序逻辑的敏感性。例如，给定图 A.6中提出的 FSM，我们可以提出关于协议开始接收第一个数据流时事件的问题：第 3 个时钟上升沿及其后续一个时钟周期发生了什么？答案将从 FSM 进行分析：在第 3 个上升沿， $state = S_0$ 。然而，在后续时钟周期中 $in = 0$ 通过组合逻辑触发 $next_state$ 变为 S_1 。然后在第 4 个时钟上升沿， $state$ 通过时序逻辑被采样为 S_1 。类似地，我们可以询问第一个数据流传输完成时发生的事情：请解释 $data_cnt = 9$ 时的状态转换。

除了问题类型外，我们还考虑了问题格式。例如，我们将一些陈述性问题改为判断题或选择题形式，以增加问题的多样性。

A.4.2.2 生成具体时序图

由于 FSM 的状态转换严重依赖输入信号 in ，纯随机的方式生成时序图会产生不可预测的输出，难以为其分配 QA 对。为了解决这一挑战，我们开发了一种自顶向下条件随机生成方法，在时序图多样性和 QA 对设计便利性之间取得平衡。

场景划分 此过程将事务过程分为不同场景。如图 A.5所示，我们将 FSM 轨迹分为两组：成功（A）和失败（B）。

钳位插入 在随机分配输入之前，我们在输入序列中添加固定钳位，如图 A.5(A) 和 (B) 中的带区 a、c、d、f、g 所示。带区 a 表示“110”序列，确保状态在第 4 个时钟上升沿变为 S_1 以开始位传输。我们在带区 c 和 f 中分配 1 用于奇偶校验。

随机生成 图 A.5(A) 和 (B) 中的带区 b、e、h 是我们分配条件随机输入的位置。例如，对于图 A.5(A) 和 (B) 中的带区 b，其序列必须包含偶数个 1，使得带区 b+c 中 1 的个数为奇数。相反，我们要求带区 e 中的序列包含奇数个 1，使得带区 e+f 中 1 的个数为偶数。

模式复现 通过钳位和条件随机输入，我们可以在时序图中复现固定模式。例如，由于带区 b+c 中 1 的个数为奇数，我们可以在图 A.5(A) 和 (B) 的带区 k 中观察到 odd 的高脉冲。相反，带区 l 中 odd 的值为低，因为检测到偶数个 1。此外，在图 A.5(A) 和 (B) 中，带区 d、k 和带区 g、l 的连锁反应将分别导致带区 i 和带区 j 中的固定结果。带区 i、j、k、l 中的稳定模式有助于 QA 对的分配。

A.4.2.3 为抽象时序图生成数据

对于抽象时序图，我们重点关注常用的 AMBA AXI[30]、AHB[31] 和 APB[32] 协议，以及 ADC[1] 的接口设计，这些在嵌入式系统中被广泛使用。这些协议的手册包含多个时序图，展示了关键场景，如 APB 中的读/写传输和 AHB 中的突发传输。从这些资源中，我们选择了若干时序图并为其手动制作 QA 对。这些 QA 对专门针对常见的工程挑战而设计。鉴于这些抽象时序图的规模较小，我们采用了轻量级随机化方法来改变其呈现方式，包括更改信号顺序、变化时钟周期数和改变字母大小写。

A.4.2.4 统计

表 A.1 各设计任务的问题类型数量

任务	类型数	任务	类型数
serial_datapath_stop	24	motor_timer_failure	13
serial_datapath_wait	24	complete_fsm	13
serial_parity_stop	22	fancy_timer	16
serial_parity_wait	22	sync_fifo	15
HDLC_correct	13	async_fifo	14
HDLC_discard	12	spi_no_clk	25
HDLC_error	13	spi_with_clk	33
w_counter	10	ADC	12
motor_timer_success	13	AXI/AHB/APB	16

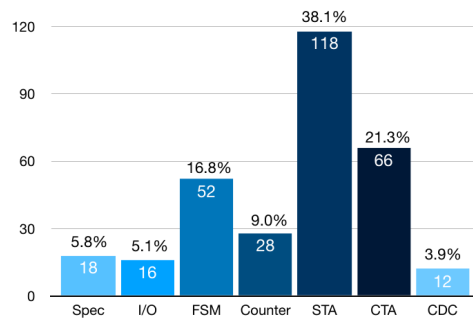


Fig. 7: Question Distribution

图 A.7 问题分布

表 A.1展示了我们考虑的所有设计任务。总共设计了 310 种不同类型的问题。图 A.7进一步展示了它们的分布。一些任务具有特定的问题类型；对于“async_fifo”

案例，如图 A.1 的第二个示例所示，被问到最多的问题与跨时钟域（CDC）相关。我们还发现大多数问题来自简单时序分析（STA）和复杂时序分析（CTA）组。STA 问题需要解释单个信号或单个时钟周期的事件，而 CTA 问题需要回答不同信号之间的关系。对于实际设计案例，如“spi_no_clk”（从机无时钟）和“spi_with_clk”（从机有独立时钟），复杂度显著升级，因此大多数问题属于 CTA 组。在第 A.6 节中，我们提供了几种问题类型的具体示例。

A.5 TD-Interpreter

A.5.1 架构

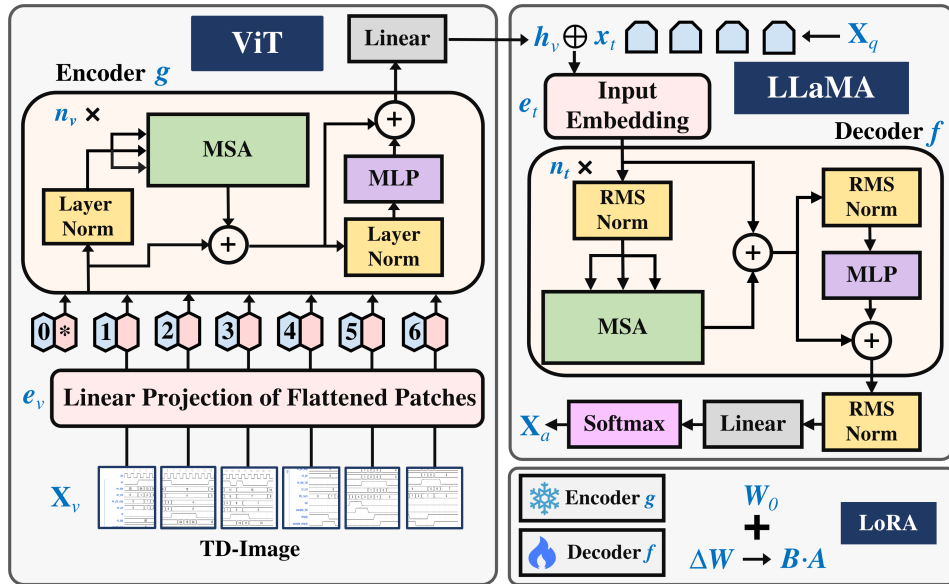


图 A.8 TD-Interpreter 的模型架构

如图 A.8 所示，TD-Interpreter 是使用我们的数据集开发的微调 LLaVA 模型 [2]。它包含一个基于视觉 Transformer（ViT）[19] 的视觉编码器 $g(\cdot)$ ，使模型能够理解图像信息，以及一个语言模型 $f(\cdot)$ ，基于视觉输入和文本请求生成文本响应。TD-Interpreter 以时序图图像 $X_v \in \mathbb{R}^{H \times W \times C}$ 和文本问题 X_q 作为输入，其中 $H \times W$ 表示 X_v 的分辨率， $C = 3$ 表示时序图图片中的 RGB 通道数。视觉编码器 $g(\cdot)$ 首先从输入图像中提取视觉特征 $h_v = g(X_v)$ 。随后，语言模型基于视觉嵌入 h_v 和从 X_q 转换的文本输入 token 序列 x_t ，预测目标文本响应 X_a 的下一个 token，表示为 $e_t = f(h_v, x_t)$ 。

A.5.1.1 视觉编码器

基于 ViT[19] 的视觉编码器 $g(\cdot)$ 由 n_v 个相同的 Transformer 块 $g_i(\cdot)$ ($1 \leq i \leq n_v$) 以及一个输入嵌入层 $e_v(\cdot)$ 和一个输出线性层组成。每个块包含一个多头自注意力 (MSA) 层、一个多层感知机 (MLP) 层和两个层归一化 (LN) [33] 层。输入时序图图像 X_v 的详细编码过程可表示如下。首先, 图像 X_v 被分割为一系列展平的 2D 补丁 $x_v \in \mathbb{R}^{N \times (P^2 \cdot C)}$, 其中 $N = (H \times W)/P^2$ 为补丁数量, $P \times P$ 为每个补丁的分辨率。然后, 补丁序列由输入层 e_v 编码: $z_p^0 = e_v(x_v)$ 。

随后, 序列 z_p^0 由 Transformer 块 $g_i(\cdot)$ ($1 \leq i \leq n_v$) 处理, 其中第 i 个块 $z_p^i = g_i(z_p^{i-1})$ 的编码过程可表示为:

$$\hat{z}_p^{i-1} = \text{MSA}(\text{LN}(z_p^{i-1})) + z_p^{i-1} \quad (\text{A.1})$$

$$z_p^i = \text{MLP}(\text{LN}(\hat{z}_p^{i-1})) + \hat{z}_p^{i-1} \quad (\text{A.2})$$

最终, 通过视觉编码器的最后线性层对 $z_p^{n_v}$ 进行变换, 得到视觉特征 $h_v = \text{Linear}(z_p^{n_v})$ 。

A.5.1.2 语言模型

语言模型 $f(\cdot)$ 是基于 LLaMA 架构 [8] 的预训练大语言模型 (LLM), 由 Vicuna[34] 模型参数化 (如文献 [2] 所述)。具体而言, $f(\cdot)$ 由一个输入嵌入层 $e_t(\cdot)$ 、 n_t 个相同的 Transformer 块 $f_i(\cdot)$ ($1 \leq i \leq n_t$) 以及一个包含 RMSNorm (RN) [35] 层和线性层的输出块组成。给定从请求 X_q 转换的文本输入 token 序列 x_t , 首先由输入层 $e_t(\cdot)$ 编码:

$$z_t^0 = e_t(\text{Concatenate}(x_t, h_v)) \quad (\text{A.3})$$

然后, z_t^0 由 n_t 个 Transformer 块编码, 其中第 i 个块 $z_t^i = f_i(z_t^{i-1})$ 的编码过程可表示为:

$$\hat{z}_t^{i-1} = \text{MSA}(\text{RN}(z_t^{i-1})) + z_t^{i-1} \quad (\text{A.4})$$

$$z_t^i = \text{MLP}(\text{RN}(\hat{z}_t^{i-1})) + \hat{z}_t^{i-1} \quad (\text{A.5})$$

最终, 预测的 logits 可表示为 $h_t = \text{Linear}(\text{RN}(z_t^{n_t}))$ 。通过对 h_t 应用 softmax 并在每个解码步骤中提取最可能的 token, 得到目标文本响应 X_a 的预测下一个 token。

A.5.2 训练

如第 A.4 节所详述，对于每个时序图图像 X_v ，我们构建一个单轮对话数据 (X_q, X_a) 。然后，对于长度为 l 的目标答案序列 X_a ，其概率计算为：

$$p(X_a|X_v, X_q) = \prod_{i=1}^l p_{\theta}(x_i|X_v, X_q, X_{a,<i}) \quad (\text{A.6})$$

其中，我们固定视觉编码器 $g(\cdot)$ 的参数，而语言模型 $f(\cdot)$ 的参数保持可训练，记为 θ 。此外， $X_{q,<i}$ 和 $X_{a,<i}$ 分别表示当前第 i 个 token 之前的问题和已生成答案的 token。

为了提高训练速度和效率，我们在 TD-Interpreter 的所有层上采用了低秩适应 (LoRA) [36] 技术并进行微调。直觉上，LoRA 引入了一个参数量远少于原始权重的可训练适配器。通过仅训练轻量级适配器而非完整的权重矩阵，LoRA 大幅减轻了训练负担。具体而言，给定预训练权重矩阵 $W_0 \in \mathbb{R}^{d \times k}$ ，其中 $d \times k$ 为矩阵的形状，LoRA 将权重矩阵从 W_0 修改为 $W_0 + \Delta W = W_0 + B \cdot A$ ，其中 $A \in \mathbb{R}^{r \times k}$ 和 $B \in \mathbb{R}^{d \times r}$ 是两个可训练的低秩矩阵，且 $r \ll \min(d, k)$ 。该技术有效地将每层的可训练参数从 $d \times k$ 减少到 $r \times (d + k)$ 。

A.6 实验评估

A.6.1 训练细节

我们首先准备了一个评估数据集作为测试基准。该数据集涵盖了表 A.1 中几乎所有的使用案例，还包含了图 A.7 中代表信号间各种因果关系的所有问题类型。对于每个问答对，由于随机生成，所有时序图图片都不相同，且在训练过程中从未出现过。最复杂的图片包含多达 51 个信号和 80 个时钟周期。如第 A.4 节所述，这些数据先由人工创建，然后自动化用于随机生成，因此对这些数据的评估能够体现模型在不同实际使用案例和问题类型上的性能。

对于训练数据集，尽管使用我们在第 A.4 节描述的数据生成器能够生成数百万条合成数据，但我们发现没有必要训练全部数据量，而且这对于计算资源有限的研究机构或企业来说也不经济。我们通过经验发现，当总训练数据量达到约 10,000 条时即可获得理想的评估结果。因此，对于基于描述的数据，我们随机采样了 4,942 个时序图图片，每个配一个 QA 对；对于基于推理的数据，我们随机采样了 5,292 个时序图图片，每个配一个 QA 对。随后我们在 8 块 NVIDIA A800 GPU 上训练模型，批量大小为 32。训练过程耗时 17 小时，共 100 个 epoch。学习率为 10^{-4} ，LoRA 秩设为 8。LoRA 适配器包含 2000 万参数。

A.6.2 实验结果

由于视觉问答任务的输出为文本，真实标签也以文本形式给出，我们可以使用常用的文本比较指标（如 Bleu-x 和 Rouge-x）来评估模型输出的准确性。“-x”代表计算特定指标的不同方法。Bleu-x 和 Rouge-x 的取值范围在 0 到 100 之间，其中 100 表示模型输出与标注答案完全重叠，0 表示最差情况。本文仅关注 Bleu-x 和 Rouge-x 的绝对值。

我们首先评估了未经微调的 LLaVA，得到以下结果：Bleu-4=13.9，Rouge-1/2/l=23.2/6.6/16.3。相比之下，微调后，LLaVA 取得了令人期待的成绩：Bleu-4=95.9，Rouge-1/2/l=96.7/95.9/96.5。这些对比结果具有双重意义：（1）微调前，LLaVA 的时序图理解能力非常有限，微调后性能有巨大提升；（2）我们观察到微调后，LLaVA 在所有 Bleu-x 和 Rouge-x 指标上均超过 95 分。这意味着对于评估数据集中的所有问题，LLaVA 都能正确回答，仅有少量小错误。

以下给出四个说明性示例，展示 TD-Interpreter 的能力。在每个问题编号后，我们标注了其对应于图 A.7 的问题类型。我们选择了相对复杂的问题，如来自 FSM、CDC 和 CTA 组的问题，以展示 TD-Interpreter 在复杂时序关系中的推理能力。部分问题因源自特定设计问题而需要非常规的答案。我们也将结果与 GPT-4o 进行了比较。蓝色文字标示了 GPT-4o 犯错的地方。此外，我们提供了人类反馈以反映设计者可能面临的典型问题。

A.6.2.1 AHB 协议

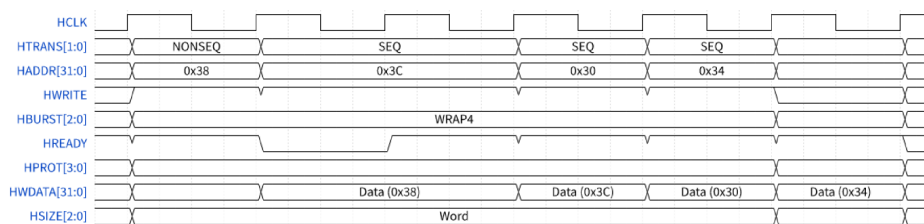


图 A.9 AHB 突发写传输

图 A.9 中的时序图是使用高级高性能总线（AHB）规范的突发写传输示例。

Q1 (CTA): 为什么写入地址 0x3C 比其他传输花费更长时间？

A1 (TD-Interpreter): 写入地址 0x3C 实际上并没有花费更长时间。相反，写入地址 0x38 花费了更长时间。其中一个从设备在此传输中添加了一个等待状态，由 HREADY 信号为低电平指示。这延长了其后续传输的地址阶段，在此情况下即为写入地址 0x3C 的传输。总之，传输到地址 0x38 的数据阶段包含一个等待状态，延长了传输到地址 0x3C 的地址阶段。

A1 (GPT-4o): GPT-4o 注意到写传输期间 HREADY 信号为低。然而，它未能

看到这是由于前一个传输期间插入的等待状态造成的。随后它错误地将写入 0x3C 识别为突发中的第一个传输。

人类反馈：在我们的实证研究中，五位声称熟悉 AHB 协议的参与者尝试分析该时序图。然而，只有 1 人正确分析了 Q1 所问的时序行为，而大多数参与者简单地说“不知道”。平均信心水平也很低，仅为 2.2/5（1 表示不自信，5 表示非常自信）。

A.6.2.2 花式定时器

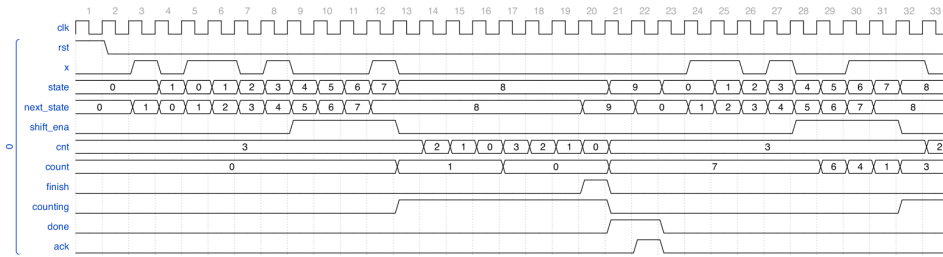


图 A.10 花式定时器

图 A.10 中的时序图展示了一个花式定时器模块，满足以下要求：（R1）检测到特定模式（1101）时启动；（R2）再移入 4 位以确定延迟持续时间；（R3）等待计数器完成计数；（R4）通知用户并等待用户确认定时器。

Q1 (FSM)：你能综合一个 Moore FSM 吗？

A1 (TD-Interpreter)：提供了一个完整的 FSM：

- (R1): $s_0 \xrightarrow{x} s_1, s_0 \xrightarrow{\sim x} s_0, s_1 \xrightarrow{x} s_2, s_1 \xrightarrow{\sim x} s_0, s_2 \xrightarrow{x} s_2, s_2 \xrightarrow{\sim x} s_3, s_3 \xrightarrow{x} s_4, s_3 \xrightarrow{\sim x} s_0$
- (R2): $s_4 \xrightarrow{\sim rst} s_5, s_4 \xrightarrow{rst} s_0, s_5 \xrightarrow{\sim rst} s_6, s_5 \xrightarrow{rst} s_0, s_6 \xrightarrow{\sim rst} s_7, s_6 \xrightarrow{rst} s_0, s_7 \xrightarrow{\sim rst} s_8, s_7 \xrightarrow{rst} s_0$
- (R3): $s_8 \xrightarrow{finish} s_9, s_8 \xrightarrow{\sim finish} s_8$
- (R4): $s_9 \xrightarrow{ack} s_0, s_9 \xrightarrow{\sim ack} s_9$
- 输出: $s_4 \sim s_7 \rightarrow \text{shift_ena} = 1, s_8 \rightarrow \text{counting} = 1, s_9 \rightarrow \text{done} = 1$

A1 (GPT-4o)：只提供了非常高层次的 FSM 设计。

Q2 (计数器)：在哪个时钟周期“cnt”和“count”都等于 0，为什么在下一个时钟周期定时器立即停止工作？

A2 (TD-Interpreter)：看第 20 个时钟周期。在这个周期中，“cnt”和“count”都等于 0。这可以用作触发信号将“finish”设为 1。由于当前状态为 s_8 ，根据 FSM，如果在此状态下 $\text{finish} = 1$ ，则 next_state 变为 s_9 。然后在第 21 个时钟上升沿，state 被采样为 s_9 。这将禁用信号“count”并启用信号“done”。

A2 (GPT-4o): 未能定位“cnt”和“count”两个信号值都等于0的正确时钟周期。

人类反馈: 我们没有要求参与者分析该时序图, 因为即使对人类专家来说, 从该时序图综合FSM也需要大量时间。然而, 如第A.3节所述, 从波形生成FSM是实际需求, 因此当TD-Interpreter提供此功能时, 将为工程师节省大量时间。

A.6.2.3 异步 FIFO

图A.1(B)中的时序图展示了深度为4的异步FIFO模块。

Q1 (CDC): 以下关系是否正确? $\text{grayencode}(\text{wr_exp}) \rightarrow \text{rd_clk} \rightarrow \text{wr_exp_r} \rightarrow \text{wr_clk} \rightarrow \text{wr_exp_cross} \rightarrow \text{wr_clk} \rightarrow \text{wr_exp_cross_r} \rightarrow \text{wr_exp_cross_trans} = \text{graydecode}(\text{wr_exp_cross_r})$ 。

A1 (TD-Interpreter): 不正确, 正确的关系应该交换rd_clk和wr_clk。

A1 (GPT-4o): GPT-4o有时认为提供的关系是正确的, 有时认为是错误的, 但倾向于提供不正确的解决方案。

Q2 (CDC): 观察第一次读取操作并回答: (1) wr_num 是否代表 FIFO 内部数据的真实数量? (2) rd_num 是否代表 FIFO 内部数据的真实数量?

A2 (TD-Interpreter): (1) 否。“wr_num”工作在“wr_clk”域中, 因此在连续读取操作发生的场景下, “wr_num”无法反映FIFO内部数据的真实数量。(2) 是。从时序图中, $\text{rd_num} = \text{wr_exp_cross_trans} - \text{rd_exp}$ 。“rd_exp”在“rd_clk”域中没有信息滞后, 但“wr_exp_cross_trans”有。然而, 在读取操作之前, “wr_exp_cross_trans”已经达到正确值4并在读取期间保持不变。因此, 在这种情况下“wr_exp_cross_trans”是正确的。

A2 (GPT-4o): 在大多数情况下“rd_num”不能反映FIFO内部的数据数量, 但异步FIFO中展示的特定情况是一个例外。GPT-4o简单地提供了否定答案和教科书式的解释, 且其提出的“rd_num”计算方法是错误的。

人类反馈: 在我们的实证研究中, 六位参与者尝试分析该时序图。首先, 他们都同意可能会在图A.1(B)中提出这些问题以获得对该异步FIFO模块的基本理解。然而, 当涉及识别跨时钟域(CDC)问题时, 这些参与者犯了错误, 尽管FIFO被认为是数字设计者的基础知识。例如, 对于Q1, 只有两人答对, 而其余四人甚至没有头绪。对于Q2, 六人中有四人正确回答了“rd_num”; 对于“wr_num”, 情况更糟, 只有一人答对。

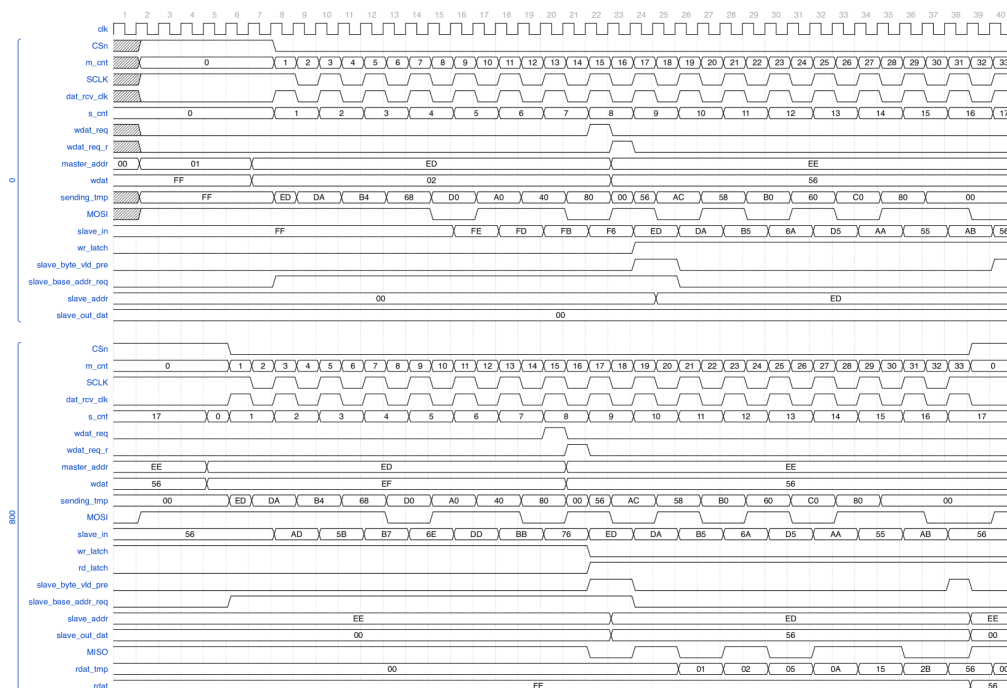


图 A.11 完整的 SPI 协议（从机无内部时钟）

A.6.2.4 SPI 协议

图 A.11 中的时序图展示了一个完整的 SPI 协议，包含主机模块、从机模块和测试平台。

Q1 (CTA): 请描述该协议的事务过程。

A1 (TD-Interpreter): (1) 主机写入：测试平台首先从主机内存的地址 X 读取数据 D，然后将数据 D 写入从机内存的地址 X。(2) 从机反馈：测试平台从从机内存的地址 X 取出数据并发回主机，以检查写入操作是否正确。

A1 (GPT-4o): 只描述了 SPI 协议的通用事务过程，忽略了测试平台在主机和从机之间数据传输中的特殊角色。

Q2 (CTA): 主机中的“sending_tmp”如何从测试平台获取地址 X 和数据 D? “MOSI”如何顺序传输位?

A2 (TD-Interpreter): 当“m_cnt”从 0 变为 1 时，“sending_tmp”从“master_addr”读取地址 X。当“m_cnt”从 16 变为 17 时，通过捕获“wdat_req_r”的下降沿，主机可以将“wdat”中的数据锁存到“sending_tmp”。“MOSI”在“SCLK”的每个下降沿传输“sending_tmp”的位。

A2 (GPT-4o): 在“m_cnt”从 16 变为 17 时，错误地捕获了“wdat_req”的下降沿而非“wdat_req_r”。

Q3 (CTA): 从机内部的“MISO”信号如何从测试平台中取出“slave_out_dat”中的数据?

A3 (TD-Interpreter) : 这发生在从机反馈期间。在“s_cnt”等于 9 时“dat_rcv_clk”的下降沿处,“slave_out_dat”开始从从机内存的地址 X 检索数据 D。然后“MISO”可以利用 s_cnt=9 的后半段持续时间通过组合逻辑采样“slave_out_dat”的最高位。接着,“MISO”可以使用 s_cnt=10 的整个持续时间处理第二高位,直到 s_cnt=17 的前半段,“MISO”采样“slave_out_dat”的最低位。由于从机没有内部时钟,“MISO”必须使用组合逻辑。

A3 (GPT-4o): 信号“MISO”通常由时序逻辑驱动,但这里的 SPI 协议表明使用组合逻辑来驱动“MISO”信号更为合理。然而,GPT-4o 仍然坚持使用时序逻辑。

人类反馈: 在我们的实证研究中,四位参与者尝试分析该时序图。尽管 SPI 是嵌入式系统中的基础外设协议,当参与者被要求评估该时序图的复杂度时,平均评分达到 3.5/5 (1 表示非常简单,5 表示非常复杂),表明他们对许多细节感到困惑。我们因此推断大多数人仅在高层次使用过 SPI,因为当他们不得不处理 Q1 所问的具体事务过程时,他们对解释需求的评分为 3.5/5 (1 表示不需要,5 表示非常需要)。我们进一步询问他们是否能够识别 A1 中提到的 X 和 D 的具体值,只有一人答对。有趣的是,只有一半的参与者发现从机模块缺少内部时钟可能导致问题,这意味着从机模块必须由组合逻辑驱动。

A.6.3 讨论与局限性

从实验中,我们观察到 GPT-4o 在分析时序图内部的详细信息方面面临挑战。对于某些特定的设计场景,GPT-4o 通常提供通用的教科书式答案,而没有进行逐案分析。这证明了 TD-Interpreter 的有用性和有效性。

本文的一个局限性在于采用了 WaveDrom 的时序图绘制风格,这是由于其简洁性和广泛的使用基础。我们的主要关注点在于使 MLLM 能够解释时序图。因此,开发支持更多样化时序图表示的前端工具是有必要的。此外,由于资源限制,我们无法构建更大规模的数据集。然而,本工作最重要的贡献在于为行业(特别是其 AI 赋能部门)提供了一种非常具有表现力的、保护知识产权的低成本方法。此外,使用 MLLM 帮助工程师理解时序图无疑将吸引众多研发团队。在数据集构建中集成后端验证以及通过强化学习改进 MLLM 的推理能力,都是有前景的未来研究方向。

A.7 结论

我们介绍了 TD-Interpreter，这是一个通过在我们生成的时序图和问答对数据集上微调 LLaVA 而得到的多模态大语言模型。TD-Interpreter 能够以图片形式接收时序图，并回答关于设计和验证的自然语言问题。我们展示了 TD-Interpreter 在丰富的任务集上超越了 GPT-4o，同样也超越了未经微调的 LLaVA 模型。

致谢

本项目获得了奥地利 FFG 和欧洲关键数字技术联合体（KDT JU）的资助（资助协议号 101097300）。本项目还得到了芯片联合体和奥地利研究促进署（FFG）通过 AIMS5.0 项目（欧盟资助协议号 101112089）的部分支持。

参考文献

- [1] Texas Instruments. ADC128S022 datasheet[EB/OL]. 2023. <https://www.ti.com/lit/ds/symlink/adc128s022.pdf>.
- [2] LIU H, LI C, WU Q, et al. Visual instruction tuning[J]. Advances in Neural Information Processing Systems, 2023, 36.
- [3] LIU M, PINCKNEY N, KHAILANY B, et al. VerilogEval: Evaluating large language models for Verilog code generation[C]//Proceedings of ICCAD. IEEE, 2023: 1-8.
- [4] LIU M, et al. ChipNeMo: Domain-adapted LLMs for chip design[A]. 2023.
- [5] THAKUR S, et al. Benchmarking large language models for automated Verilog RTL code generation[C]//Proceedings of DATE. IEEE, 2023: 1-6.
- [6] THAKUR S, et al. VeriGen: A large language model for Verilog code generation[J]. ACM Transactions on Design Automation of Electronic Systems, 2024, 29(3): 1-31.
- [7] NVIDIA. LLM4HWDesign[EB/OL]. 2024. <https://nvlabs.github.io/LLM4HWDesign/>.
- [8] TOUVRON H, et al. LLaMA 2: Open foundation and fine-tuned chat models[A]. 2023.
- [9] NIJKAMP E, et al. CodeGen: An open large language model for code with multi-turn program synthesis[A]. 2022.
- [10] FU Y, et al. GPT4AIGChip: Towards next-generation AI accelerator design automation via large language models[C]//Proceedings of ICCAD. IEEE, 2023: 1-9.
- [11] CHANG K, et al. Natural language is not enough: Benchmarking multi-modal generative AI for Verilog generation[A]. 2024.
- [12] HE J, NIČKOVIĆ D, BARTOCCI E, et al. TD-Magic: From pictures of timing diagrams to formal specifications[C]//Proceedings of DAC. IEEE, 2023: 1-6.

- [13] Static timing analysis[M]//Advanced ASIC Chip Synthesis Using Synopsys Design Compiler Physical Compiler and PrimeTime. Boston, MA: Springer US, 2002: 261-303.
- [14] MALER O, NICKOVIC D. Monitoring temporal properties of continuous signals[C]//International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems. Springer, 2004: 152-166.
- [15] FISLER K. Timing diagrams: Formalization and algorithmic verification[J]. Journal of Logic, Language and Information, 1999, 8(3): 323-361.
- [16] AMLA N, EMERSON E A, NAMJOSHI K S. Efficient decompositional model checking for regular timing diagrams[C]//LNCS: Vol. 1703 Proceedings of CHARME. Springer, 1999: 67-81.
- [17] BARTOCCI E, MATEIS C, NESTERINI E, et al. Survey on mining signal temporal logic specifications[J]. Information and Computation, 2022, 289: 104957.
- [18] CHEN Z, et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks[C]//Proceedings of the IEEE/CVF CVPR. 2024: 24185-24198.
- [19] DOSOVITSKIY A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[C]//ICLR. OpenReview.net, 2021.
- [20] Google Team, et al. Gemini: A family of highly capable multimodal models[A]. 2023.
- [21] HDLBits. HDLBits[EB/OL]. 2024. https://hdlbits.01xz.net/wiki/Problem_sets.
- [22] EDAUtils. Verilog testbench generator[EB/OL]. 2024. <https://www.edautils.com/VlogTBGen.html>.
- [23] WILLIAMS S. Icarus Verilog compilation system[EB/OL]. 2024. <https://github.com/steveicarus/iverilog>.
- [24] Namake Nanamaru. vcd2json[EB/OL]. 2024. <https://github.com/naname/vcd2json>.
- [25] WaveDrom. wavedrom-cli[EB/OL]. 2024. <https://github.com/wavedrom/cli>.
- [26] DeepSeek-AI, et al. DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence[A]. 2024.
- [27] LIU J, et al. Generated knowledge prompting for commonsense reasoning[C]//Proceedings of ACL. 2022: 3154-3169.
- [28] WEI J, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Proceedings of NeurIPS. 2022.
- [29] HDLBits. Fsm_serialdp[EB/OL]. 2024. https://hdlbits.01xz.net/wiki/Fsm_serialdp.
- [30] AMBA AXI and ACE protocol specification[M]. Arm, 2011.
- [31] AMBA AHB protocol specification[M]. Arm, 2021.
- [32] AMBA APB protocol specification[M]. Arm, 2021.
- [33] BA J L. Layer normalization[A]. 2016.
- [34] CHIANG W L, et al. Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality[J]. See <https://vicuna.lmsys.org>, 2023, 2(3): 6.
- [35] ZHANG B, SENNRICH R. Root mean square layer normalization[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [36] HU E J, et al. LoRA: Low-rank adaptation of large language models[A]. 2021.

书面翻译对应的原文索引

参考文献

- [1] HE J, KENBEEK V T W, YANG Z, et al. TD-Interpreter: Enhancing the understanding of timing diagrams with visual-language learning[J]. arXiv preprint arXiv:2507.16844, 2025.

致 谢

五年前，那个顶着酷暑、拖着行李箱沿新民路走过紫操的汤秉达，大概怎么也想象不到：今天深夜，他会坐在电脑前，一边敲着论文致谢，一边神经紧绷地盯着实验进度——更想象不到，今天的他，未来还将走向何方。

彼时刚从高考的硝烟中抽身，他自然不会天真地以为大学之路会一帆风顺，却仍怀揣着一种朴素而炽热的期待：期待自己像故事里的勇者那样，纵然历经重重险阻，仍能以革命乐观主义精神披荆斩棘；期待友情、爱情、兴趣、学业与科研齐头并进，在青春最丰盛的年岁里谱写一部“王道故事”；期待一段崭新的、灿烂的、带着玫红色光泽的校园时光。

然而，世界不是玫红色的，而是各种乱七八糟的颜色混杂在一起。那么，汤秉达当然只能度过一段乱七八糟的岁月，在跌跌撞撞间有所斩获，亦有所缺憾。幸运的是，尽管今天的他对自己尚不完全满意，却也邂逅了许多意想不到的惊喜，在某些地方悄悄突破了昔日所能设想的边界。感慨的是，哪怕今天的他已经决定与过往握手言和（真的吗？笑），当年的他恐怕依然难以置信：自己竟会趟过这般煎熬，留下这许多遗憾。

他总忍不住思索：如果在某个“当初”做了另一种选择，是否会有另一种可能？但毕竟人终究无法将希望寄托于“其他的可能性”这无从知晓的东西，他只能接受此刻的他，这个也哭也笑平凡着的他，明白他不会成为除此之外的任何人。

回过头来，他觉得自己还是应当感到格外庆幸：在无数条不可知的世界线中，这唯一收束为现实的一种，倒也并不算坏——甚至可以说颇具几分传奇色彩。因此，他并不后悔当年的选择，也从未后悔与那些人共同度过的时光。他挺起胸膛，将一切引以为傲。

这五年间他所经历的所有精彩，除却自身的选择与命运的偶然之外，更仰仗于许多曾与他的生命轨迹相交、共鸣的人们。所以，他想把那些名字写下来，那些让他走到此刻、走到这一页的人。于是他写道：

时间令记忆变得无关紧要，令所有辉煌的东西失重。但在岁月的沉淀中，我依然清楚地记得：这一路上曾向我伸出援手的师长与同辈。

感谢郑舒文学长、李逸飞学长、罗关学长、严若天学长、刘郭越学姐、左一翔学长、刘弘显学姐以及来源老师等师长，在我尚于建院求学期间为我答疑解惑，尤其是在转专业那段迷茫的时期。你们给予的帮助与鼓励给了我莫大的前行动力。

感谢罗峻骁学长，引领我第一次深度涉足科研，第一次以共同一作的身份发

表学术论文；在这段旅程中，我不仅收获了宝贵的科研经验，更收获了宝贵的友谊。也感谢陈挺老师的指导与支持，以及在博士申请阶段提供的推荐。

感谢赛宁老师在纽约大学暑研期间给予我的指导与支持，以及对我博士申请的帮助。您严谨的治学态度与深刻的学术见解，令我受益匪浅。感谢郑博阳与潘禧辰学长对论文工作的鼎力相助，没有你们的协力，论文难以顺利付梓。

感谢张钰晖学长、汪晓晗学长以及 Serena Yeung-Levy 老师在斯坦福大学暑研期间给予的指导与支持，以及在申请季提供的宝贵推荐。

行至本科生涯的尾声，感谢喻文健老师、何卓论老师对我毕业设计的耐心指导以及童圣博学长在毕设期间的帮助，为我的本科岁月画上了圆满的句号。

感谢于新雨、花佳诚、王玮康、汪隼立、方行健等同学在求学路上的相携相助，让我的学习生活更加顺遂充实。感谢李星汉、徐思涵、金伟阳、张凯文、李赫、吕博涵等好友，与我一路探讨博士申请的难关。感谢章俊一学长、Colin 学长、连龙学长、李博学长、吴鹏浩学长、高信学长等师长的真知灼见，帮助我抉择博士 Offer。感谢 Trevor Darrell 老师给予的宝贵机会，亦感谢刘子纬老师的赏识。

才识是岁月的冠冕，正如思念是我们共度的时间。感谢所有在这段旅程中包容和支持我的朋友与亲人。

感谢陪伴我到今天的执信人。特别感谢陈思铭、金胜昔。无须多言，愿我们久经考验的友谊历久弥新。感谢王格、童尧、张邹灵、欧子晴，在他乡求学的日子里，老同学的相聚总能扫去疲惫。感谢刘聪、徐有成等学长曾给予的诸多关照，也感谢刘付羽亭、戴思巍学姐、熊楚珺在生活中的关心。

感谢军乐队的每一位伙伴。感谢李旭娜，让我结识了军乐队。感谢康书博学长无私的指导，让我从零起步，领略了圆号的魅力。感谢彭茜婷学姐、王翰毅、张晴朗、唐天舒、周洋豫、韩昊成学长等圆号声部的朋友们。感谢李星汉、孙宇尧、吴宇迪、李岷洲。与你们相聚，便无话不可倾诉，愿 1597 的友谊长青。感谢李嘉芃、郑睿阳、封羽真、宋昊原等“发电群”的大家，我们在一段旋律里暂别，必将在另一段旋律重新相遇。

感谢吉柄旭、赵恒、胡跃彬，我们在紫十一共同度过了和谐而愉快的五年。感谢李晨宇、郑博阳，我们在 Aquablu 一起感受了纽约的繁华与喧嚣。感谢张凯文、封羽真，我们一起享受了湾区的阳光与海风。

最后，要致以最深切谢意的，是我的父母。感谢你们一直以来无条件支持与鼓励，让我能心无旁骛、安心地去追寻自己的理想与热爱。

同时，也感谢一路跋涉、未曾轻言放弃的自己。辛苦了。

行文至此，难免仓促，若有疏漏未及致谢之处，还请多多包容。

声 明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____ 日 期：_____